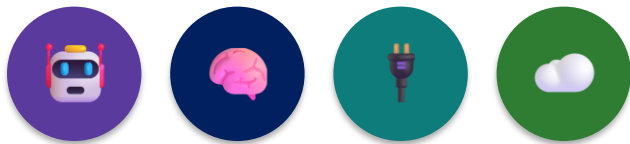


DAY 5

OpenShift AI, MCP, and Capstone Completion

Risk Assessment Service + OpenShift AI integration; capstone demonstration. · Lab 5 — Risk Assessment Service

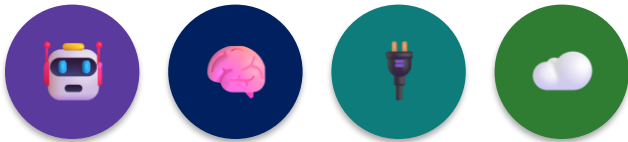




MODULE 12

Copilot, OpenShift AI, and MCP

Use Copilot, OpenShift AI, and MCP as force multipliers on the same architecture: generate, ground, and govern so banking code stays reviewable.



INTRODUCTION TO GITHUB COPILOT · GITHUB COPILOT PRODUCT CAPABILITIES



WHAT COPILOT HELPS BUILD

- REST controllers, service layer methods, repository interfaces, DTO classes
- Validation rules, unit tests, refactoring suggestions, API documentation



THE KEY MESSAGE

- Generated code must still meet enterprise standards for security, correctness, and compliance
- Copilot is a development assistant, not a replacement for software design



CORE CAPABILITIES

- Complete code, generate methods, create tests
- Explain unfamiliar code, suggest refactoring, generate documentation, draft configuration files

KEY TAKEAWAY Copilot generates boilerplate, tests, and documentation, but engineering judgment, review, and testing still belong to the developer. • Understanding the full capability surface helps a team decide which Copilot mode fits which part of the Spring Boot workflow.

HOW AI CODING ASSISTANTS GENERATE SUGGESTIONS · GITHUB COPILOT IN THE IDE

· COPILOT CHAT AND INLINE ASSISTANCE



WHAT SHAPES A SUGGESTION

- Existing project files and open editor tabs
- Comments, method names, class names, chat prompts, and selected code
- The quality of suggestions depends heavily on context and prompt clarity



IDE-INTEGRATED WORKFLOW

- Suggestions appear inline as ghost text while coding
- Accept, reject, or cycle through alternative suggestions without leaving the editor

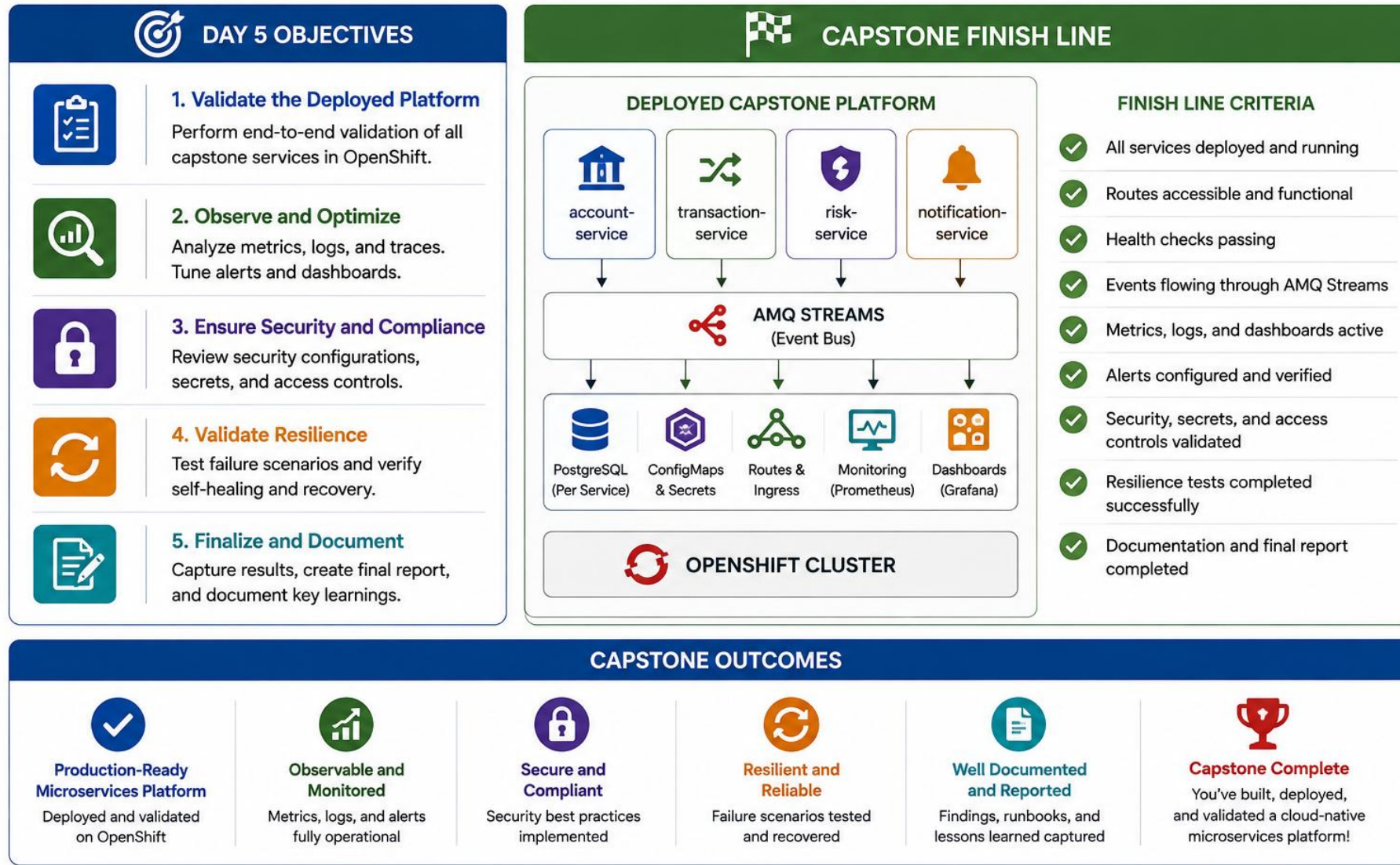


TWO INTERACTION MODES

- Copilot Chat: conversational, multi-turn requests for larger code units or explanations
- Inline assistance: fast, in-editor suggestions as you type, accepted with a keystroke

KEY TAKEAWAY Existing project files, open editor tabs, comments, method and class names, and chat prompts all shape what Copilot suggests next. • Copilot proposes code directly at the cursor as you type, accepted or rejected line by line or block by block. • Chat suits larger, conversational requests with back-and-forth refinement; inline completion suits fast, in-flow micro-suggestions.

Day 5 Objectives and Capstone Finish Line



UNDERSTANDING REPOSITORY CONTEXT · PROMPT ENGINEERING FOR SOFTWARE DEVELOPMENT



WHY REPOSITORY CONTEXT MATTERS

- Copilot infers project conventions from surrounding code, not just the current file
- Keeping related files open improves the relevance of generated suggestions

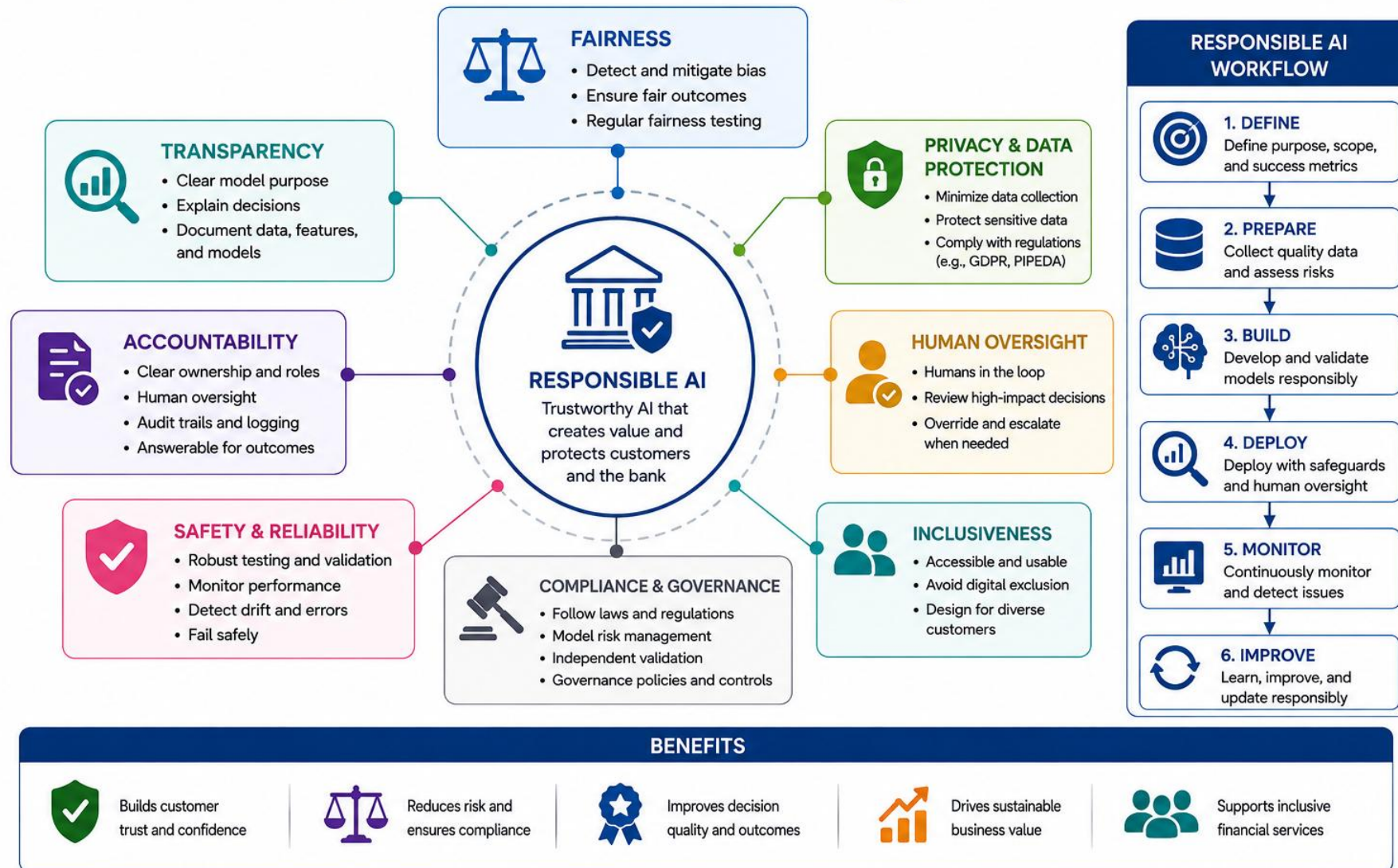


WHAT GOOD PROMPTS DESCRIBE

- The framework, the class or method needed, existing project patterns
- Required inputs and outputs, validation rules, error handling and testing expectations

KEY TAKEAWAY A repository with consistent naming, layering, and DTO conventions gives Copilot the strongest signal to follow those same conventions. • Good coding prompts describe the framework, the class or method needed, existing patterns, inputs/outputs, validation, and error handling.

Responsible AI in Banking Workflows



WRITING CLEAR CODING PROMPTS · PROVIDING CONSTRAINTS AND ACCEPTANCE CRITERIA

VAGUE PROMPT

- "Create customer API." -- no endpoints, DTOs, validation, or status codes specified

BETTER PROMPT

- Name the controller, base path, methods, DTOs, @Valid usage, and expected status codes
- State explicitly: "Do not put business logic in the controller. Follow constructor injection."

PROMPT COMPONENTS

- Role: "Act as a Spring Boot developer"
Task: "Create a controller"
- Context: existing DTOs and exception handling
Constraints: "Use constructor injection"
- Output: "Return only the Java class"
Quality: "Include validation and HTTP status codes"

KEY TAKEAWAY A vague prompt leaves endpoints, validation, and status codes to chance; a specific prompt gives Copilot enough structure to draft correctly. • Role, task, context, constraints, output format, and quality expectations are the six ingredients of a well-formed coding prompt.

USING EXAMPLES AND EXISTING CODE AS CONTEXT · ITERATIVE PROMPTING AND REFINEMENT



WHY EXAMPLES WORK

- A concrete example transmits conventions that are hard to fully describe in words
- Pointing Copilot at an existing, well-reviewed class as a pattern improves consistency



THE ITERATIVE LOOP

1. Generate initial code
2. Review gaps
3. Ask for improvements
4. Add missing constraints
5. Refine to match project standards

KEY TAKEAWAY An example of an existing controller or service teaches Copilot your naming, layering, and error-handling conventions by demonstration. • Generate initial code, review gaps, ask for improvements, add missing constraints, and refine to match project standards -- five iterative steps.

GENERATING A SPRING BOOT PROJECT STRUCTURE · GENERATING REST CONTROLLERS · GENERATING SERVICE-LAYER LOGIC

SCAFFOLDING A NEW SERVICE

- Prompt for the full package structure: controller, service, repository, model, dto, exception
- Review that generated classes land in the correct package for their responsibility

WHAT TO SPECIFY IN THE PROMPT

- Collaborators (repository, event publisher), method signatures, business rules
- Exceptions to throw, @Transactional placement, event publication timing

EXPECTED CONTROLLER SHAPE

- @RestController, base @RequestMapping, method mappings, DTO request/response types
- @Valid validation trigger, @PathVariable/@RequestBody, appropriate ResponseEntity usage

COMMON GENERATED PROBLEMS

- Missing business rules, incorrect exception handling, events published before save succeeds
- Forgotten @Transactional, excessive duplicated mapping code

COMMON GENERATED PROBLEMS

- Business logic in the controller, entities returned directly, missing @Valid
- 200 OK returned for create instead of 201 Created; invented service methods

KEY TAKEAWAY Describing the target package layout up front keeps Copilot's generated classes landing in the right layer from the very first draft. • @RestController, DTOs, @Valid, and correct status codes define a good generated controller; business logic in the controller is a common defect. • Prompting for exceptions, transaction annotations, and event publication order explicitly prevents Copilot from guessing at business rules.

GENERATING REPOSITORY COMPONENTS · GENERATING DTOS AND VALIDATION RULES · GENERATING EXCEPTION HANDLING



EXPECTED REPOSITORY SHAPE

- extends JpaRepository<Entity, Long>, Optional for single-record lookups
- findBy... derived methods and existsBy... checks matching real entity fields



PROMPTING FOR DTOS

- Name the request and response DTO fields explicitly
- State validation rules: required fields, format constraints, ranges, date rules



COMMON GENERATED PROBLEMS

- Field names that don't exist, overly broad methods, missing pagination, missing Optional

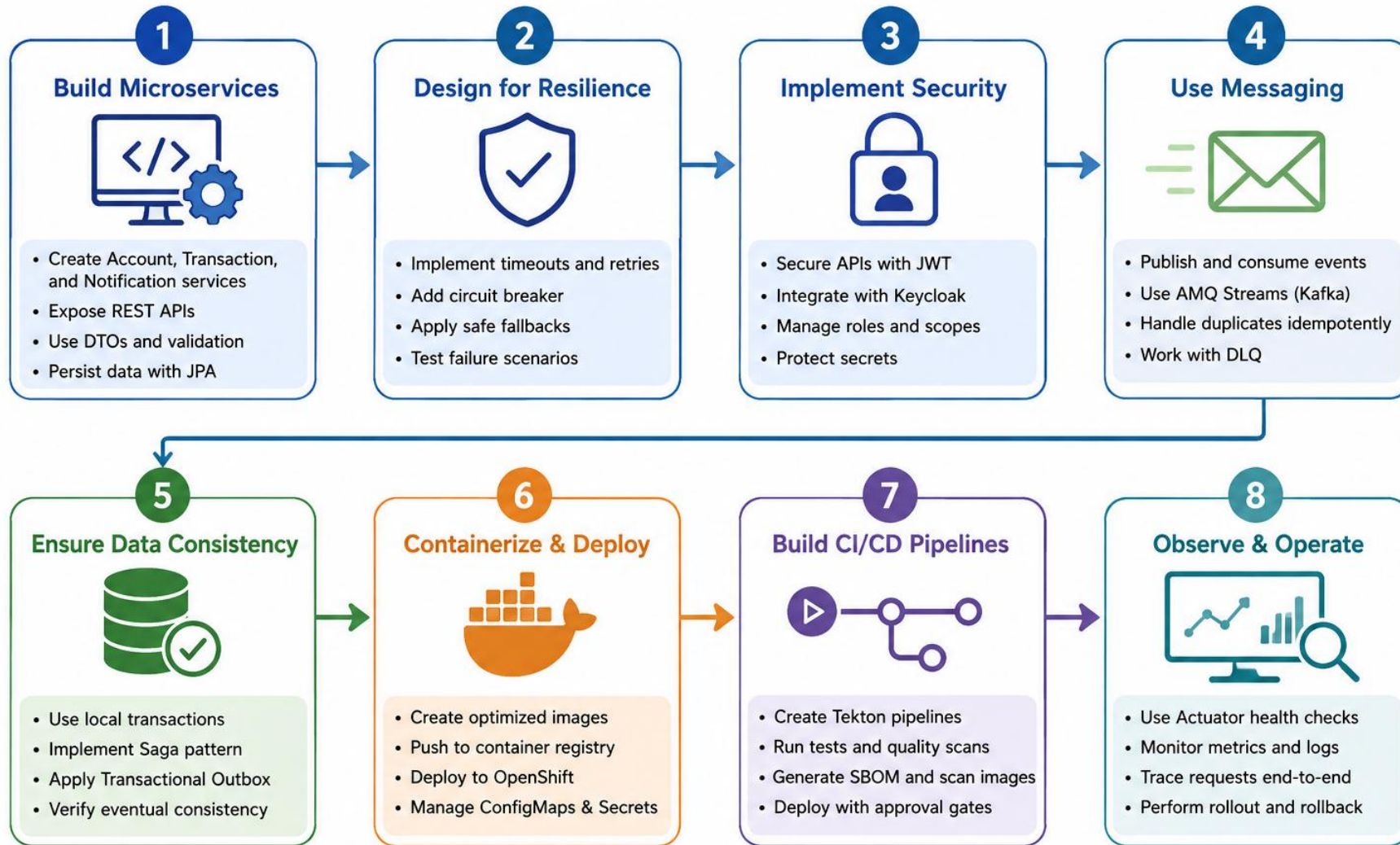


GENERATING EXCEPTION HANDLING

- Custom exception classes (e.g., CustomerNotFoundException) extending RuntimeException
- @RestControllerAdvice with @ExceptionHandler methods mapping to correct HTTP status codes

KEY TAKEAWAY Providing entity field names in the prompt is what keeps Copilot's derived query methods from referencing properties that don't exist. • @NotBlank, @Email, @Size, and @Past map cleanly onto request DTO fields once the business rules are stated in the prompt. • Custom exceptions plus one global exception handler give every generated controller the same consistent, reviewable error contract.

What Participants Will Do Hands-On



GENERATING CONFIGURATION COMPONENTS · GENERATING DATABASE QUERIES · GENERATING MESSAGING PRODUCERS AND CONSUMERS



GENERATING CONFIGURATION

- @ConfigurationProperties classes, application.yml sections, profile-specific overrides
- Never include real secrets, passwords, or environment-specific URLs in the prompt itself



PROMPTING FOR CUSTOM QUERIES

- "Create a @Query method to find recent transactions by accountId and status, ordered by date descending, returning Page<T>"
- Review generated SQL or JPQL behavior and confirm indexes exist for the queried fields



GENERATING MESSAGING CODE

- Publisher components using RabbitTemplate or KafkaTemplate with a defined event DTO
- @RabbitListener or @KafkaListener consumer methods with duplicate-event handling reviewed manually

KEY TAKEAWAY Asking for a configuration class with clear property names is safe; asking for real credentials or environment values in the prompt is not.

- Requesting a paginated, ordered @Query method explicitly avoids Copilot generating an inefficient or overly long derived query name.
- Copilot can draft a RabbitTemplate publisher or @KafkaListener consumer, but duplicate-handling logic must be reviewed, not assumed.

GENERATING UNIT TESTS · GENERATING INTEGRATION TESTS · GENERATING TEST DATA AND EDGE CASES



PROMPTING FOR UNIT TESTS

- "Generate JUnit 5 and Mockito tests for CustomerService.createCustomer: success, duplicate email, repository save failure"
- Verify mock interactions (repository, event publisher) with clear test method names



GENERATING INTEGRATION TESTS

- Controller tests validating status codes: 201 Created, 400 Bad Request, 404 Not Found
- Repository tests validating persistence behavior against a real or embedded database



PROMPTING FOR EDGE CASES

- Ask explicitly for negative opening balance, duplicate account number, missing customer ID
- Add missing edge cases manually when the generated set only covers the happy path

KEY TAKEAWAY Prompting for specific scenarios (success, duplicate, not-found) produces far stronger tests than a generic "write tests for this class" request. • A generated integration test that mocks the very layer it should be testing gives a false sense of coverage. • Negative values, boundary conditions, duplicate keys, and missing required fields rarely appear unless the prompt asks for them by name.

USING COPILOT FOR DEBUGGING · USING COPILOT FOR CODE EXPLANATION



USING COPILOT WHEN DEBUGGING

- Paste the stack trace or failing test output and ask for likely causes
- Ask Copilot to explain unfamiliar framework behavior surfacing in the error

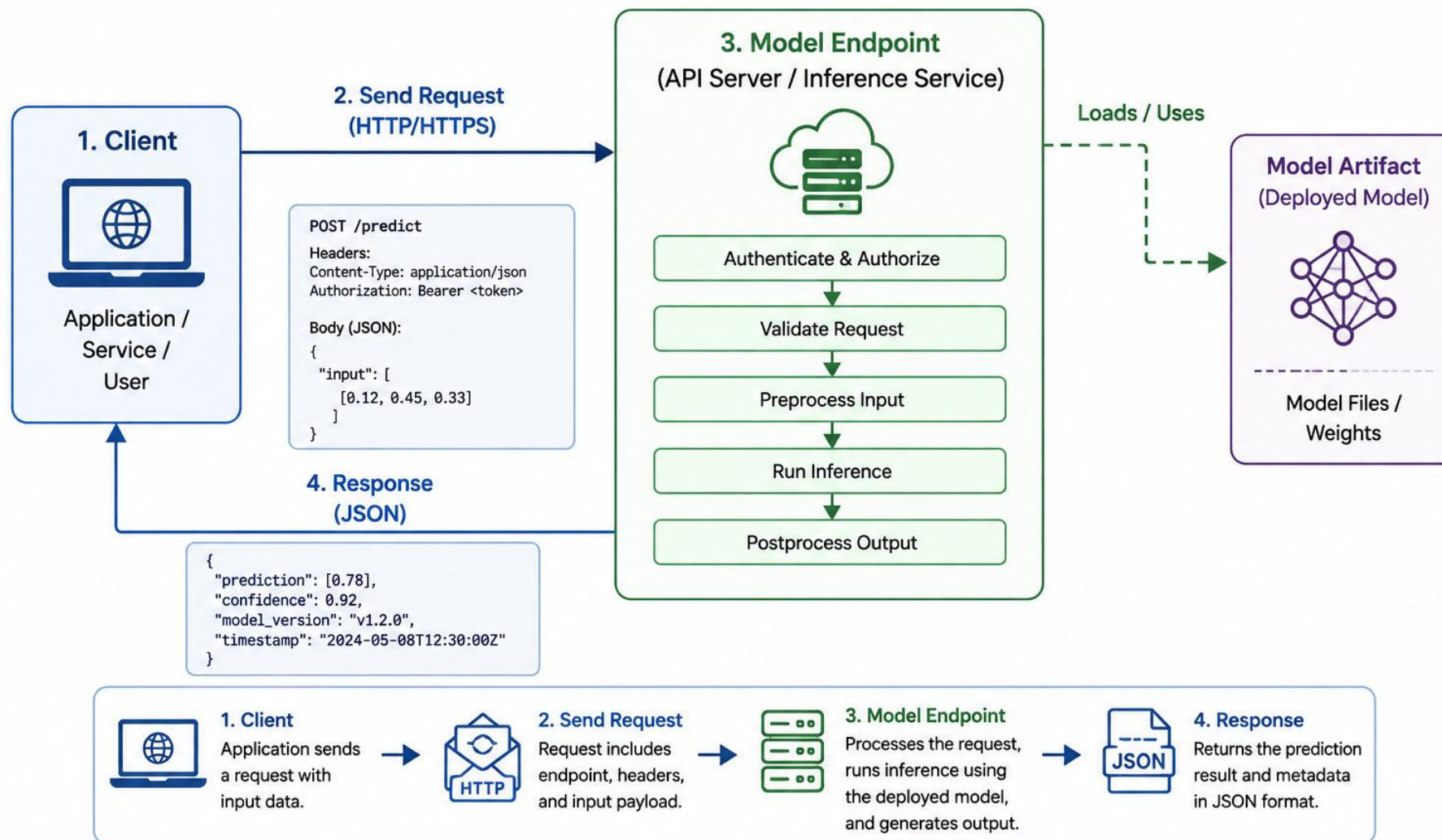


USING COPILOT FOR EXPLANATION

- Ask what a Spring annotation, design pattern, or unfamiliar method actually does
- Cross-check the explanation against official documentation for anything safety-critical

KEY TAKEAWAY Pasting an exception and asking "why might this happen" is often faster than searching, but the suggested fix still needs validation against real behavior. • A clear explanation of an annotation or pattern is a starting point for understanding, not a substitute for verifying it against the real codebase.

Calling a Model Endpoint



USING COPILOT FOR REFACTORING · IMPROVING PERFORMANCE WITH COPILOT · GENERATING API DOCUMENTATION



SAFE REFACTORING WORKFLOW

1. Ensure tests exist
2. Select a focused code section
3. Ask for a refactoring plan or draft
4. Apply small changes
5. Run tests
6. Review behavior and API compatibility



PERFORMANCE-FOCUSED PROMPTS

- Ask Copilot to identify N+1 query risks, missing indexes, or unnecessary eager loading
- Measure query count and latency before and after applying any suggested change

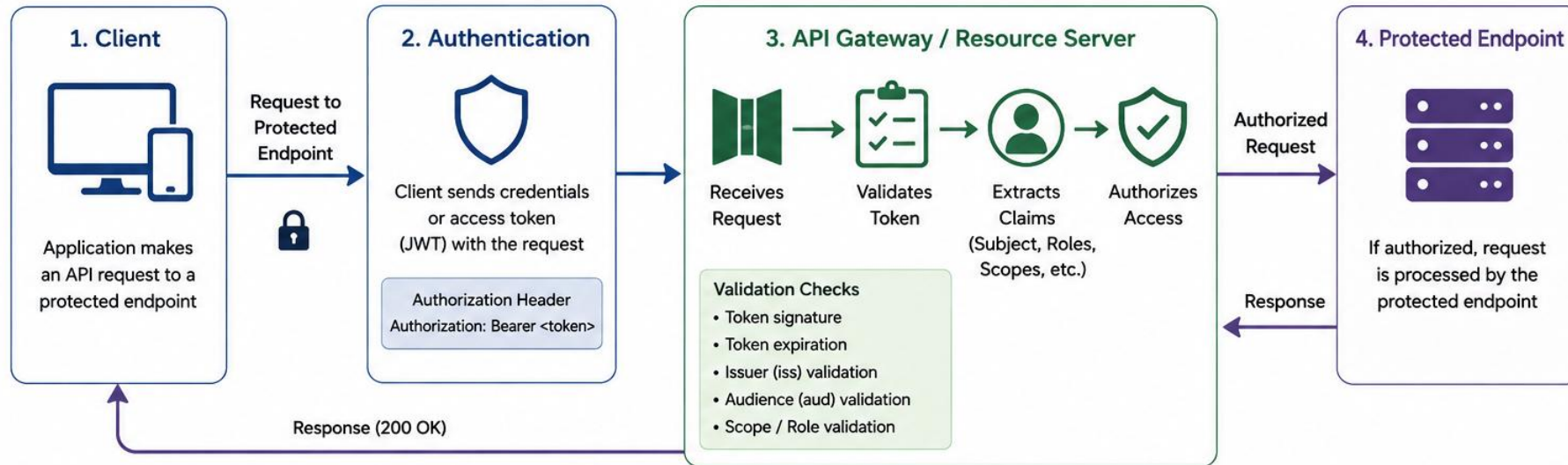


PROMPTING FOR DOCUMENTATION

- "Generate README documentation including overview, endpoints, examples, errors, environment variables, run instructions"
- Explicitly instruct: "Do not include secrets" and verify every endpoint against real code

KEY TAKEAWAY Ensure tests exist, refactor a focused section, apply small changes, and run tests -- the same discipline whether a human or Copilot writes the diff. • A performance suggestion is a hypothesis until profiling data confirms it actually improved latency or query count. • A documentation draft can include fake endpoints or stale examples unless someone confirms it against the actual running service.

Endpoint Authentication

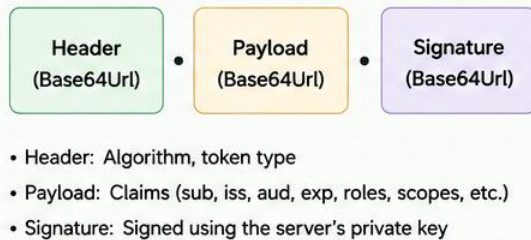


Authentication Flow (Summary)

How It Works

- 1 Client requests a protected endpoint.
- 2 Client includes a JWT (or credentials) in the Authorization header.
- 3 Resource server validates the token:
 - Verifies signature using public key
 - Checks expiration
 - Validates issuer and audience
 - Extracts claims (roles, scopes, etc.)
- 4 If the token is valid and the client is authorized, access is granted.
- 5 The protected endpoint processes the request and returns a response.

JWT Structure (Example)



Common Security Practices

- ✓ Use HTTPS for all endpoints
- ✓ Use short-lived access tokens
- ✓ Use refresh tokens securely
- ✓ Validate all token claims
- ✓ Use least-privilege roles and scopes
- ✓ Log authentication failures
- ✓ Rotate signing keys regularly

GENERATING CONTAINER AND PIPELINE FILES · REVIEWING AI-GENERATED CODE · DETECTING INCORRECT OR INVENTED APIS



REVIEWING GENERATED INFRASTRUCTURE FILES

- Confirm the base image is enterprise-approved and pinned to a specific version
- Confirm no secrets are embedded and the container runs as a non-root user



REVIEW CHECKLIST

- Correct package/class names, constructor injection, no business logic in controllers
- DTOs instead of entities, validation annotations, correct HTTP status codes
- No hardcoded secrets, no unsafe logging, tests for success and failure cases



COMMON HALLUCINATION PATTERNS

- Calling a method that does not exist on the actual class
- Referencing a library not present in the project's dependencies
- Describing a framework feature incorrectly, or generating config for the wrong version

KEY TAKEAWAY Approved base images, non-root users, and no embedded secrets are non-negotiable checks on any Copilot-drafted Dockerfile or pipeline stage. • Package names, constructor injection, DTO usage, validation, status codes, secrets, and tests -- a nine-point checklist for every generated class. • Calling a nonexistent method, referencing an unapproved library, or misdescribing a framework feature are all classic hallucination patterns.

SECURITY RISKS IN AI-GENERATED CODE · INTELLECTUAL PROPERTY AND LICENSING CONSIDERATIONS



COMMON SECURITY DEFECTS

- Hardcoded secrets, weak input validation, unsafe SQL construction
- Excessive or sensitive logging, missing authorization checks



WHAT TO VERIFY

- Is the suggested library on the enterprise's approved dependency list?
- Is its license compatible with how the software will be distributed or sold?

KEY TAKEAWAY Hardcoded secrets, weak validation, unsafe SQL construction, and excessive logging are the security defects most likely to slip through in generated code. • Verifying license compatibility and approval status before accepting a generated dependency protects the enterprise from downstream legal risk.

PROTECTING SENSITIVE DATA IN PROMPTS · ENTERPRISE GOVERNANCE FOR GITHUB COPILOT

· MEASURING COPILOT PRODUCTIVITY

NEVER INCLUDE IN PROMPTS

- Real customer data, account numbers, passwords, tokens, API keys
- Internal security details or confidential business logic where policy restricts sharing

RESPONSIBLE USE PRINCIPLES

- Human review, security awareness, testing discipline, policy compliance
- Data protection, traceability, architecture alignment

BETTER PRODUCTIVITY SIGNALS

- Time from task start to merged, reviewed pull request
- Defect rate in Copilot-assisted code versus baseline, review time per change

KEY TAKEAWAY Anything typed into a prompt can be logged or retained -- treat prompt content with the same care as a support ticket or a commit message.

- Human review, security scanning, pull request review, and CI quality gates apply to generated code exactly as they do to any other change.
- Cycle time, defect rate, and review time tell a more honest productivity story than raw suggestion-acceptance counts ever could.

COPILOT BEST PRACTICES FOR ENTERPRISE TEAMS · HUMAN REVIEW BEFORE ACCEPTING SUGGESTIONS



TEAM BEST PRACTICES

- Use AI for focused tasks; review generated code thoroughly; run tests and scans
- Avoid sensitive prompt content; follow enterprise dependency policy; keep PR review requirements



WHY HUMAN REVIEW IS NON-NEGOTIABLE

- Copilot is an assistant, not an accountable engineer
- The developer and engineering team remain responsible for final code quality

KEY TAKEAWAY A short list of team-wide habits keeps Copilot's speed advantage from ever coming at the cost of enterprise engineering standards. • Accepting a suggestion is not the same as approving it -- that decision belongs to the developer who will be accountable for it in production.

Generative AI vs. Agentic AI

From content generation to goal-driven action.

Both are powerful. They solve different problems.



Generative AI

Creates content
when prompted.



Agentic AI

Acts toward goals
autonomously.

ASPECT	GENERATIVE AI	AGENTIC AI	WHEN TO USE WHICH
Purpose	Generate content, ideas, or predictions based on input. <i>Answers questions. Creates outputs.</i>	Achieve goals by taking actions, using tools, and adapting to feedback. <i>Gets things done. Delivers outcomes.</i>	Use Generative AI when you need to: ✓ Create or summarize content ✓ Write code snippets or documentation ✓ Translate or explain complex topics ✓ Brainstorm ideas and alternatives ✓ Rapidly prototype text or images
How it works	Responds to prompts with learned patterns from training data. <i>Stateless between interactions.</i>	Plans, reasons, takes actions, observes results, and adapts. <i>Stateful, iterative, and goal-directed.</i>	
Autonomy	Human-in-the-loop for every request. No persistent autonomy. <i>You ask, it responds.</i>	Can operate with a degree of autonomy within defined boundaries. <i>You set the goal, it plans and acts.</i>	
Capabilities	Text, code, images, summaries, translation, brainstorming, Q&A, and more. <i>Great at creation.</i>	Use tools, call APIs, access data, take actions, coordinate workflows, learn from feedback. <i>Great at execution.</i>	Use Agentic AI when you need to: ✓ Achieve multi-step goals ✓ Orchestrate systems and APIs ✓ Automate business processes ✓ Monitor, adapt, and optimize ✓ Handle dynamic, real-world tasks
Examples	ChatGPT, GitHub Copilot (inline suggestions), DALL-E, Claude, Gemini. <i>Content generation tools.</i>	AutoGPT, Microsoft Copilot (agent mode), OpenAI Assistants API, LangChain agents. <i>Autonomous agents and copilots.</i>	
Strengths	• Fast, creative, and scalable content generation • Excellent for ideation and augmentation • Easy to use and prompt-driven	• Goal-oriented and outcome-focused • Can use tools and integrate with systems • Adapts to changing context and feedback	
Limitations	• No real-world actions or tool use • Can hallucinate or be factually incorrect • Requires human direction for every step	• More complex to design and govern • Higher risk if not properly bounded • Requires strong guardrails and observability	

THE CONTINUUM



THE BIG PICTURE

Generative AI helps you **create**.

Agentic AI helps you **accomplish**.

Together, they amplify human impact.



The future isn't one or the other—it's human + AI working together with the right tool for the right job.



Human Intent
Sets goals and
defines success



AI Capability
Provides intelligence
and automation



Governance
Ensures safety,
ethics, and trust



Outcome
Delivers measurable
business value

WHAT MAKES AN AI SYSTEM AN AGENT · MODELS, AGENTS, TOOLS, AND ENVIRONMENTS · AGENT GOALS AND TASK DECOMPOSITION



AGENTIC WORKFLOW ELEMENTS

- Goal, context, plan, tools, memory or state
- API access, observations, decision points, human approval, final output



THE FOUR BUILDING BLOCKS

- Model: the reasoning engine (LLM) that interprets goals and plans steps
- Agent: the orchestrating layer around the model that manages tools, memory, and policy
- Tools: callable capabilities (APIs, functions) the agent can invoke
- Environment: the system the agent operates within, including its constraints

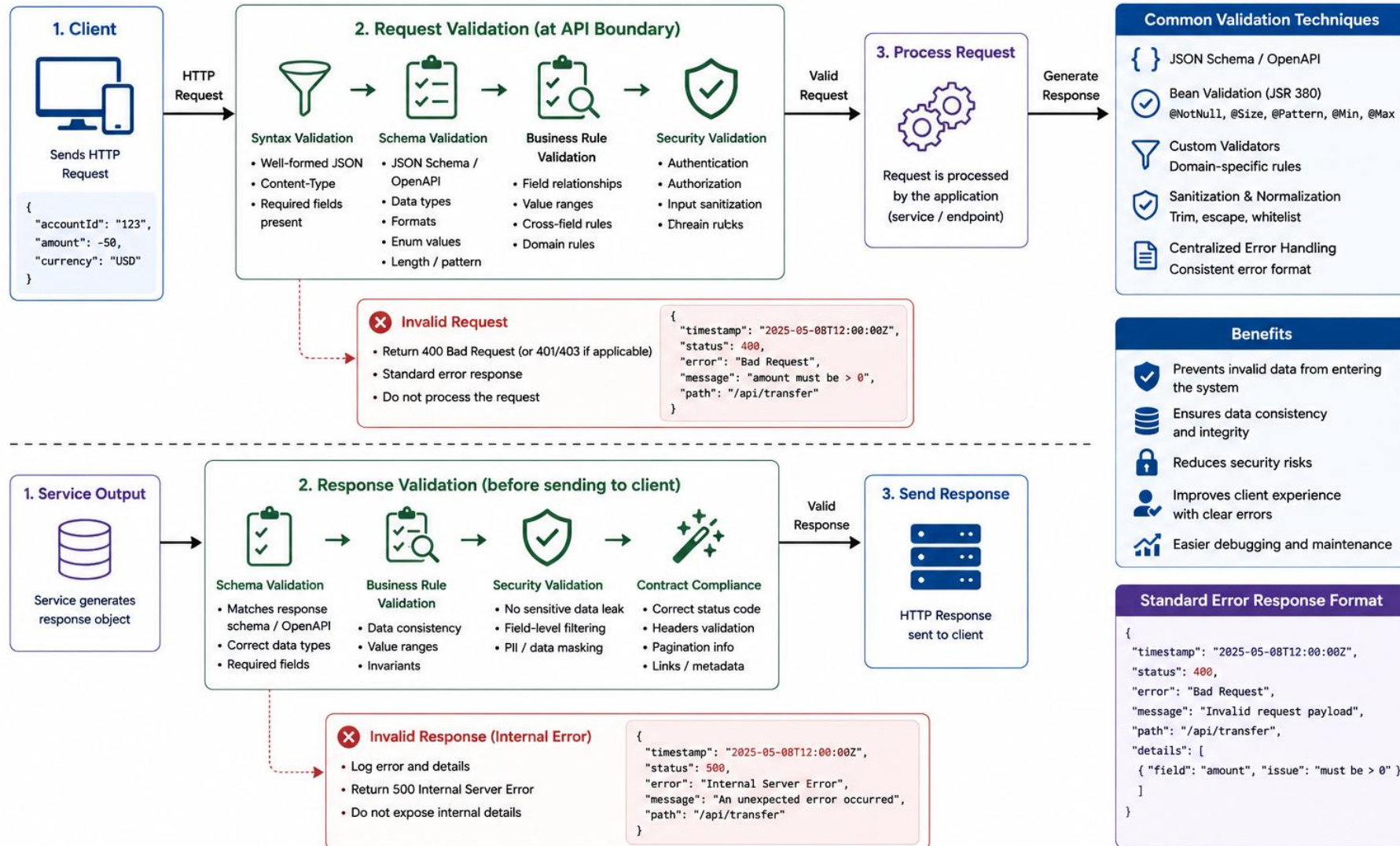


GOAL CLARITY MATTERS

- Good: "Summarize failed payment reasons for support review."
- Too broad: "Handle customer banking issues."
- A clear goal decomposes into concrete, tool-callable steps

KEY TAKEAWAY Goal, context, plan, tools, memory, and observations together turn a language model into something that can act, not just respond. • Separating these four concepts clarifies exactly where a Spring Boot API fits: as a tool the agent can call, not as the agent itself. • "Summarize failed payment reasons for support review" decomposes cleanly; "handle customer banking issues" does not.

Request/Response Validation



PLANNING AND REASONING LOOPS · OBSERVATION, ACTION, AND FEEDBACK



THE REASONING LOOP

1. Plan the next action based on the goal and current state
2. Act by calling a tool or producing output
3. Observe the result
4. Replan or conclude



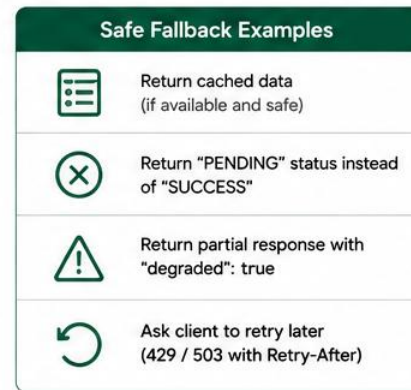
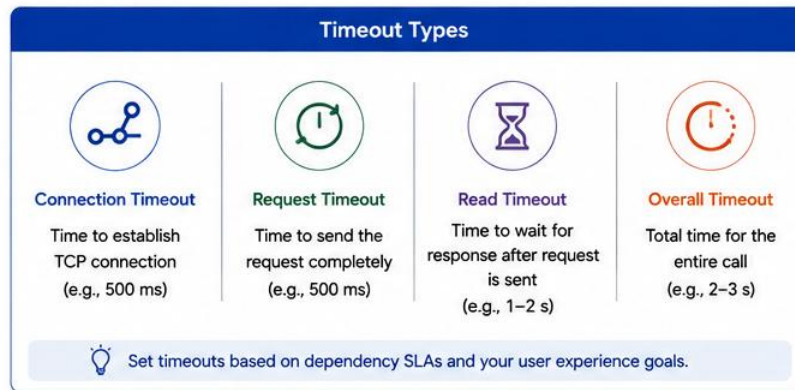
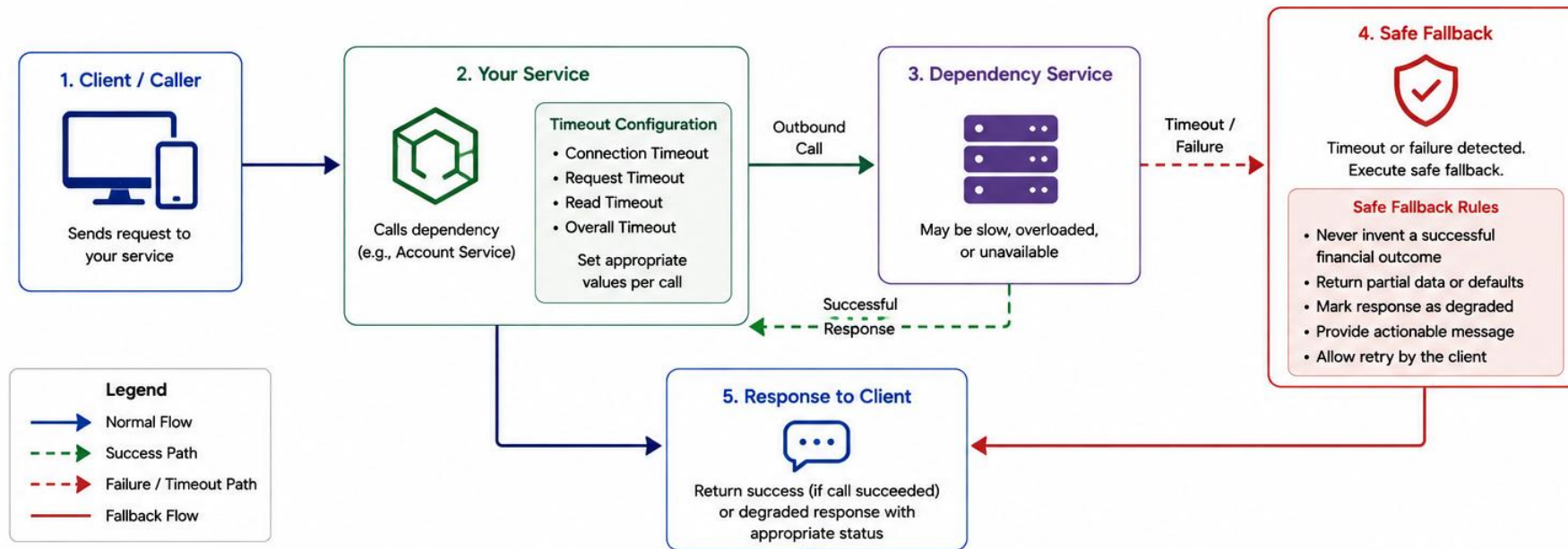
THE OBSERVE-ACT-FEEDBACK CYCLE

- Action: the agent calls a tool or takes a step toward its goal
- Observation: the agent reads the actual result, success or failure
- Feedback: the observation informs the next planning step

KEY TAKEAWAY This plan-act-observe loop is what separates a multi-step agent from a single-shot prompt-and-response interaction. • If a tool call fails or returns unexpected data, the agent must observe that honestly rather than proceeding as if it succeeded.

Timeouts and Safe Fallback

Fail fast. Degrade safely. Never invent a successful financial outcome.



TOOL CALLING AND FUNCTION CALLING · AGENT MEMORY AND CONTEXT

· SHORT-TERM VS. LONG-TERM MEMORY



HOW TOOL CALLING WORKS

- The model is given a list of available tools with names, descriptions, and parameter schemas
- The model responds with a structured request to call one, which the framework executes
- The tool's result is returned to the model to continue reasoning



MEMORY VS. CONTEXT

- Context: the information available to the model for the current reasoning step
- Memory: information retained and retrievable across steps or sessions



TWO MEMORY TYPES

- Short-term memory: the current task's working context, often the conversation window
- Long-term memory: persisted facts, preferences, or history retrievable across sessions

KEY TAKEAWAY The model doesn't execute the tool itself -- it requests a call, the agent framework executes it, and the result flows back into the conversation.

- Without memory, an agent re-asks questions it already had answers to; without bounded context, it drowns in irrelevant history.
- A support agent's short-term memory holds this conversation's transaction IDs; long-term memory might recall a customer's past support history.

RETRIEVAL-AUGMENTED GENERATION · DETERMINISTIC WORKFLOWS VS. AUTONOMOUS AGENTS



HOW RAG WORKS

- A retrieval step searches an indexed knowledge base for relevant content
- Retrieved content is added to the model's context before it generates a response
- This grounds answers in real, current, trusted source material



WHEN TO USE EACH

- Deterministic workflow: the sequence of steps is known and fixed in advance
- Autonomous agent: the model decides which tools to call and in what order, based on context
- Prefer deterministic workflows when the process is well-understood and repeatable

KEY TAKEAWAY An AI assistant retrieving relevant policy documents before answering a support question is a direct application of RAG in a banking context.

- Deterministic workflows are predictable and easy to test; autonomous agents are flexible but require stronger guardrails and observability.

SINGLE-AGENT VS. MULTI-AGENT SYSTEMS · AGENT ORCHESTRATION PATTERNS · HUMAN-IN-THE-LOOP WORKFLOWS



SINGLE-AGENT VS. MULTI-AGENT

- Single-agent: one model handles the entire task end to end -- simpler to reason about
- Multi-agent: specialized agents each handle part of the workflow and coordinate results



COMMON ORCHESTRATION PATTERNS

- Centralized orchestrator: one controller directs each agent's turn explicitly
- Decentralized coordination: agents react to shared events or state independently



WHY HUMAN-IN-THE-LOOP MATTERS

- Adds a checkpoint before sensitive or irreversible actions take effect
- Keeps a human accountable for the final decision on regulated or high-impact outcomes

KEY TAKEAWAY A multi-agent design might have one agent gather data, another summarize it, and a third draft the customer-facing response.

- The orchestration pattern chosen determines how visible, debuggable, and controllable a multi-agent workflow will be in production.
- The assistant drafts a customer message; a support representative reviews and sends it -- the agent never communicates with the customer directly.

AGENT PERMISSIONS AND BOUNDARIES · DESIGNING SAFE AUTONOMOUS WORKFLOWS · ERROR HANDLING AND RECOVERY FOR AGENTS



AGENT BOUNDARY QUESTIONS

- What can the agent read? What can the agent change? Which tools are available?
- Which actions need approval? What data is masked? What is logged?



WHAT TO DEFINE BEFORE BUILDING

- Trigger, goal, allowed tools, data access scope
- Decision rules, human approval points, error handling, stop conditions, audit logging



RECOVERY OPTIONS

- Return a draft instead of sending, escalate to human review, create a case
- Retry safe tool calls only, stop and report missing data clearly

KEY TAKEAWAY Defining these boundaries explicitly before deployment is what prevents an agent from accidentally exceeding its intended authority. • Trigger, goal, allowed tools, data access scope, decision rules, approval points, error handling, and stop conditions -- design all eight up front. • Returning a draft instead of sending, escalating to human review, or retrying only safe operations are all valid recovery paths -- silence is not.

Audit Capture Requirements

1. WHAT TO CAPTURE								
Who - User ID / Subject - Roles / Scopes - Client ID / App - Source IP Address	What - Action / Operation - Resource Type - Resource ID - Request Method - Endpoint / URI	When - Timestamp (UTC) - Timezone (if needed) - Event Date	Where - Service Name - Environment - Host / Instance ID - Zone / Region	Outcome - Result (Success / Failure) - HTTP Status Code - Error Code / Reason - Response Time (ms)	Data - Request ID / Correlation ID - Transaction ID (if any) - Amount / Currency (if applicable) - Before / After (for changes)	Security & Integrity - Digital Signature / Hash - Log Sequence / Index - Retention Class - Tamper Evidence		
2. AUDIT EVENT STRUCTURE (EXAMPLE)		3. QUALITY REQUIREMENTS		4. RETENTION & ACCESS				
<pre>{ "eventId": "01HYZ8X4Q3K3V9J1A2B3C4D5E6", "eventType": "ACCOUNT_DEBIT", "timestamp": "2025-05-08T12:34:56.789Z", "user": { "userId": "user123", "roles": ["TELLER"] }, "client": { "clientId": "web-banking", "ip": "203.0.113.10" }, "action": "DEBIT", "resource": { "type": "ACCOUNT", "id": "ACC-10001" }, "request": { "method": "POST", "uri": "/api/v1/accounts/ACC-10001/debit" }, "outcome": { "result": "SUCCESS", "httpStatus": 200, "errorCode": null }, "data": { "amount": 250.00, "currency": "USD", "correlationId": "corr-123e4567-e89b-12d3-a456-426614174000" }, "durationMs": 145, "service": { "name": "account-service", "env": "prod", "instanceId": "acct-svc-7f9ds" }}</pre>		Accuracy Events must be accurate and complete. Timeliness Capture events as close to real-time as possible. Completeness No audit events should be lost or omitted. Integrity Protect logs from alteration or deletion. Consistency Use standard formats, codes, and time (UTC).		Retention Retain audit logs per regulatory and business requirements (e.g., 7 years). Access Control Restrict access to authorized roles only. Monitoring & Review Regularly review audit logs and set up alerts for suspicious activities. Archival Archive logs securely and ensure they remain searchable.				
5. PRINCIPLES								
Accountability Every action must be traceable to a responsible entity.	Non-Repudiation Ensure events cannot be denied or altered.	Transparency Logs should be clear, understandable, and auditable.	Privacy Capture only necessary data and protect sensitive information.	Compliance Meet regulatory and industry audit requirements.				
6. EXAMPLES OF AUDIT EVENTS								
- Login / Logout - Account Create / Update - Funds Transfer / Payment - Limit Change - Beneficiary Management			- Role / Permission Change - Failed Login Attempts - Configuration Change - Data Export / Report Download - Admin Actions					

PREVENTING INFINITE AGENT LOOPS · AGENT EVALUATION AND TESTING · AGENT OBSERVABILITY



LOOP-PREVENTION CONTROLS

- Set a maximum number of reasoning/tool-call steps per task
- Detect repeated identical failures and stop instead of retrying indefinitely
- Apply an overall timeout budget for the entire workflow



WHAT TO TEST

- Normal cases, missing data, conflicting data, tool failure
- Unauthorized access, sensitive data handling, human approval flow, output format validation



WHAT TO OBSERVE

- User request, model call, tool calls, API latency, failure points
- Generated output, approval decisions, cost and token usage where applicable

KEY TAKEAWAY Maximum step counts, timeout budgets, and repeated-failure detection are the guardrails that keep an autonomous loop from running away. • An agent that has never been tested against a failing tool or an unauthorized request is an agent whose failure behavior is unknown. • If an agent's behavior can't be reconstructed after the fact, it can't be trusted to run unattended in front of real customers.

TRUST, RISK, AND RESPONSIBLE AI · PROMPT INJECTION AND TOOL ABUSE · DATA LEAKAGE AND PRIVACY RISKS



RESPONSIBLE AI PRINCIPLES

- Ground claims in evidence; disclose uncertainty rather than hiding it
- Design for graceful failure, not just for the successful path
- Match autonomy level to the real-world risk of the decision



MITIGATIONS

- Treat retrieved content as untrusted data, never as instructions
- Enforce tool permission checks independent of what the model requests



HOW TOOL ABUSE HAPPENS

- Malicious instructions embedded in retrieved documents, emails, or API responses
- An agent that trusts all input equally can be tricked into calling tools it shouldn't



REDUCING DATA LEAKAGE RISK

- Minimize the data an agent can access to only what the task requires
- Mask or exclude PII, account numbers, and credentials before they reach the model

KEY TAKEAWAY Responsible AI means designing for the case where the agent is wrong, not just for the case where it happens to be right. • Prompt injection isn't only about direct user input -- a poisoned document, email, or API response can carry the attack just as effectively. • Minimizing what data an agent can see in the first place is more reliable than trusting it to always choose not to repeat sensitive fields.

AGENTIC AI USE CASES IN MICROSERVICES · WHEN NOT TO USE AN AI AGENT · MICROSOFT AI ECOSYSTEM OVERVIEW



BANKING USE CASES

- Customer support assistance, fraud investigation triage
- Loan application pre-check, operations incident summarization
- API orchestration across customer, account, and transaction services



POOR FITS FOR AGENTIC AI

- Simple, fixed-sequence processes with no need for dynamic reasoning
- High-risk decisions where deterministic rules and audit trails matter more than flexibility
- Tasks with no available data source to ground the agent's output



THE MICROSOFT AI ECOSYSTEM

- Azure OpenAI Service, Azure AI services, Azure AI Search
- Microsoft Copilot experiences, Microsoft Graph, Azure Functions and Logic Apps
- Azure API Management, Azure Key Vault, Azure Monitor

KEY TAKEAWAY Each use case pairs a real productivity win with a defined human checkpoint before anything customer-facing or financial actually happens.

- If the steps never change and the logic is simple, a deterministic workflow is more predictable, cheaper, and easier to audit than an agent.
- Language, search, document, identity, and secrets services can all combine into one Spring Boot AI workflow architecture.

MICROSOFT COPILOT AS AN AI AGENT · AZURE AI FOUNDRY CONCEPTS · AZURE OPENAI SERVICE OVERVIEW



THE COPILOT PATTERN

- User intent -> AI assistant interprets context -> AI suggests or performs next step -> user reviews or approves



WHAT AZURE AI FOUNDRY PROVIDES

- A hub for selecting, testing, and managing foundation models
- Tooling for building, evaluating, and deploying AI agents and workflows

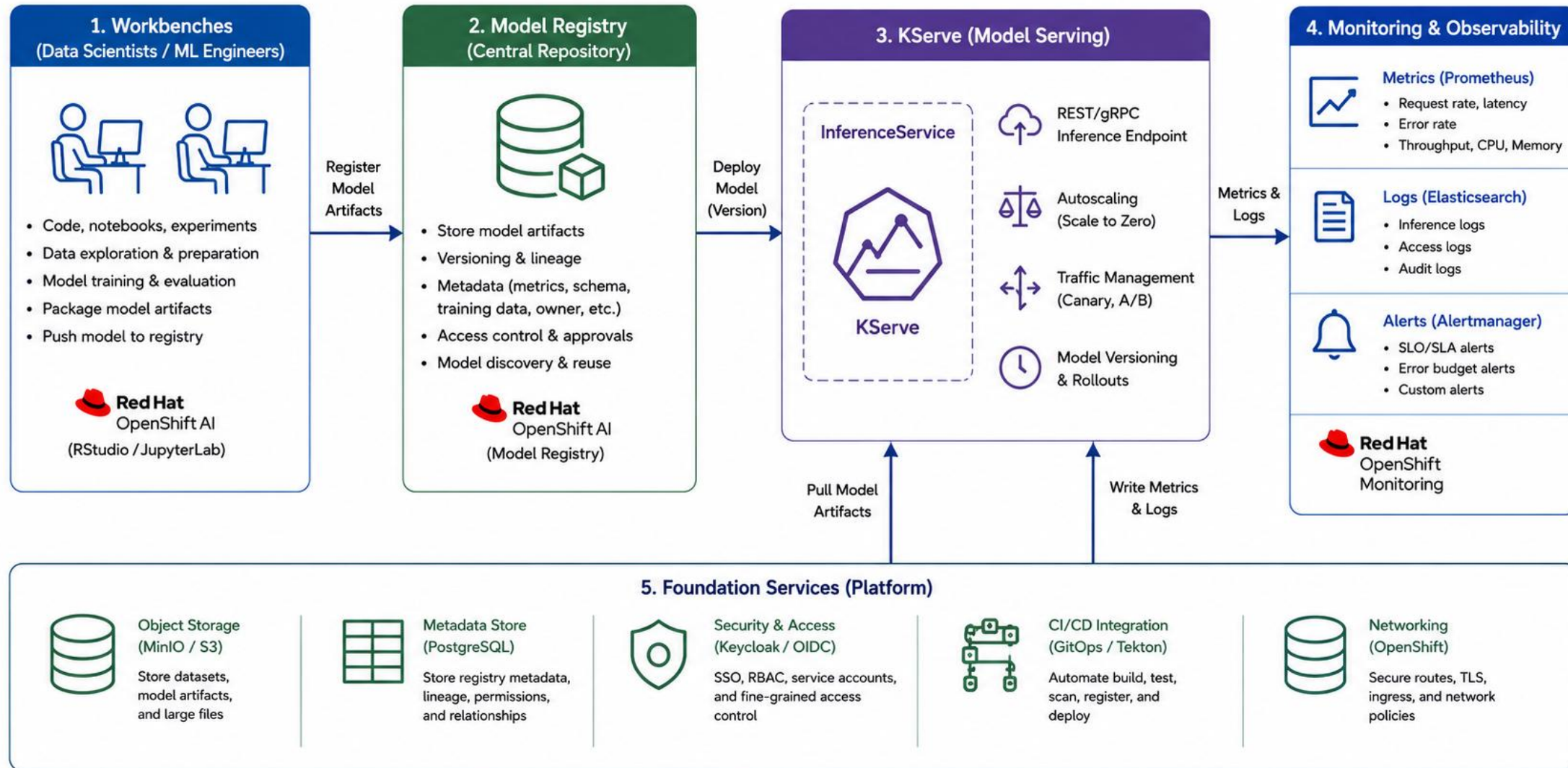


COMMON USE CASES

- Summarization, classification, draft generation
- Retrieval-augmented generation, tool orchestration, conversational assistants

KEY TAKEAWAY Whether generating code or drafting a business summary, Copilot-style assistants share one consistent shape: assist, don't autonomously decide. • A unified platform for model selection, prompt orchestration, and agent development reduces the custom glue code enterprise AI teams usually build. • Summarization, classification, draft generation, RAG, tool orchestration, and conversational assistants all run on the same managed service.

Conceptual Awareness – workbenches, model registry, KServe, monitoring



MICROSOFT SEMANTIC KERNEL OVERVIEW · AGENTS, PLUGINS, AND FUNCTIONS IN SEMANTIC KERNEL



WHAT SEMANTIC KERNEL PROVIDES

- A common abstraction for connecting to different AI models
- Plugin and function orchestration, memory management, and planning support



THE THREE LAYERS

- Agent: the orchestrating component that decides what to do
- Plugin: a grouped set of related capabilities
- Function: one specific, callable operation within a plugin

KEY TAKEAWAY It bridges the gap between calling a raw LLM API and building a fully governed enterprise agent workflow. • This layering mirrors MCP's tool model closely -- functions are narrow, named, and callable, just wired through a different SDK.

CONNECTING AGENTS TO BACKEND APIS · DEFINING TOOLS FOR MICROSOFT AI AGENTS · AI-ASSISTED API ORCHESTRATION



INTEGRATION PATTERN

- AI Assistant -> Semantic Kernel plugin
-> Spring Boot REST API
- The plugin validates input, calls the API, and maps the response back to the agent



TOOL DEFINITION ELEMENTS

- Name, purpose, input schema, output schema
- Required permissions, error behavior, data sensitivity classification

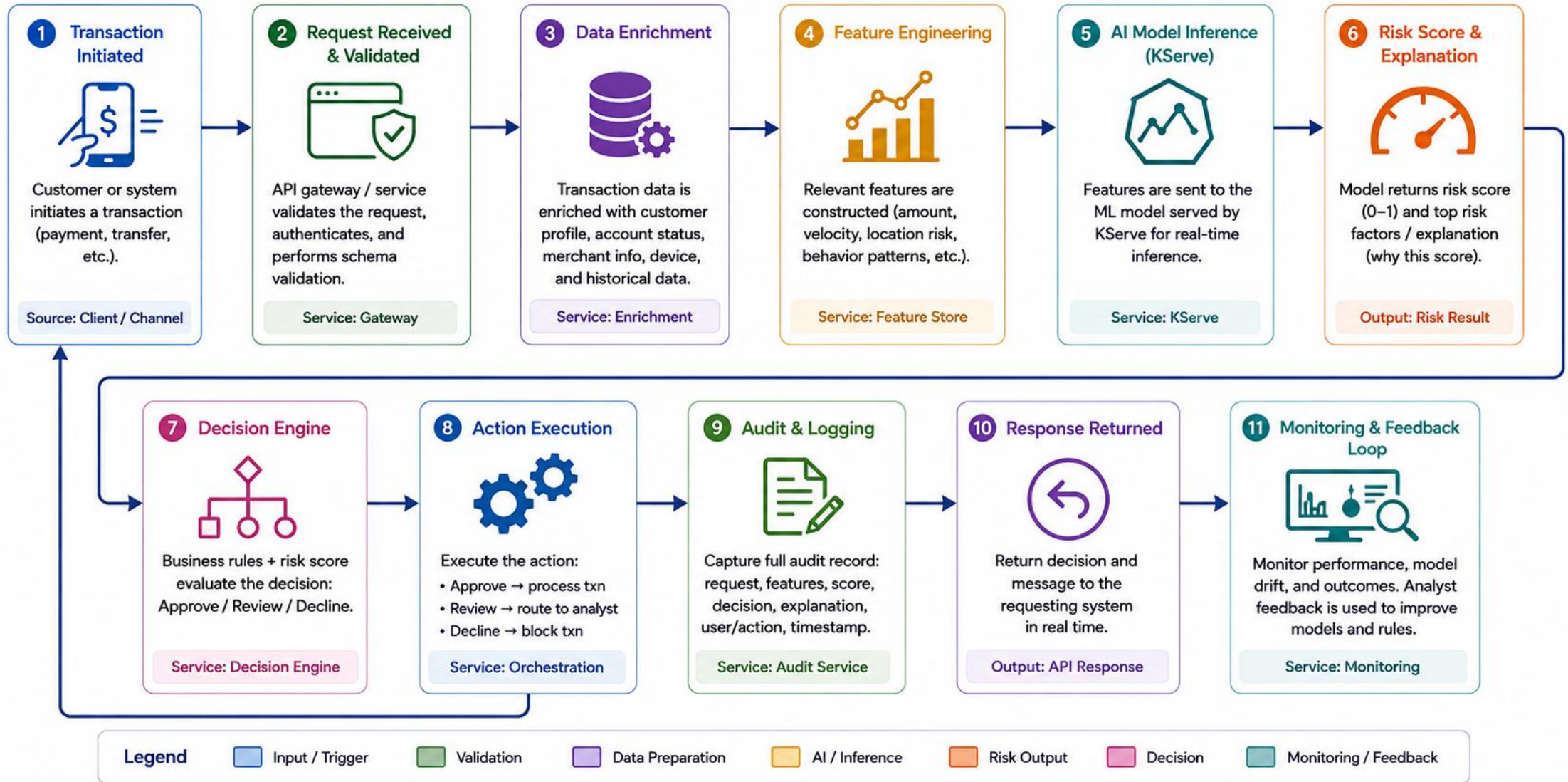


EXAMPLE ORCHESTRATION

1. Call Customer Service to confirm customer exists
2. Call Transaction Service for failed transfers
3. Call Account Service for account status
4. Call Rules Service for failure code explanation
5. Generate summary and draft response for review

KEY TAKEAWAY The agent framework changes, but the underlying discipline stays the same: call the owning service's API, never bypass it. • Whether built for Semantic Kernel or MCP, tool design principles are the same: narrow, structured, permission-aware, and safe on failure. • Confirm customer, check failed transfers, check account status, explain the failure code, then draft a response for review -- five orchestrated steps.

AI-Assisted Transaction Decision Workflow (11-step flow)



MULTI-STEP WORKFLOW DESIGN · MAINTAINING STATE ACROSS AGENT INTERACTIONS · GROUNDING AGENTS WITH ENTERPRISE DATA



DESIGNING THE WORKFLOW

- Define each step's input, output, and failure behavior explicitly
- Decide which steps can run in parallel and which must be sequential



WHERE TO STORE AGENT STATE

- Externalize workflow state to a database or cache, not local process memory
- Include a workflow or session ID that ties every step back to the same interaction



WHAT GROUNDING LOOKS LIKE

- Retrieve real data (Azure AI Search, backend APIs) before generating a response
- Include source references -- transaction IDs, policy documents -- in the final output

KEY TAKEAWAY Treating a workflow's steps as a designed sequence, not an improvised chain, is what makes it maintainable and debuggable in production. • External state stores let a workflow survive a restart or scale across multiple instances without losing where it left off. • Grounding means citing transaction IDs, account status, and policy references -- not asking the model to recall facts from memory alone.

AGENT AUTHENTICATION AND MANAGED IDENTITY · SECURING MICROSOFT AI SERVICE CONNECTIONS



WHY MANAGED IDENTITY MATTERS

- Eliminates hardcoded credentials for service-to-service Azure authentication
- Azure manages credential rotation and lifecycle automatically



SECURING THE CONNECTION

- Store API keys and endpoints in Azure Key Vault, not application.yml
- Use managed identity where supported instead of static keys

KEY TAKEAWAY No connection string, no API key in code -- Azure issues and rotates the identity's credentials automatically behind the scenes.

- Spring Boot services should retrieve AI service credentials from Azure Key Vault at runtime, exactly like a database password.

CONTENT FILTERING AND SAFETY CONTROLS · HANDLING MODEL AND TOOL FAILURES · MONITORING AI REQUESTS AND COSTS



WHAT CONTENT FILTERING COVERS

- Screens for hate speech, violence, self-harm, and sexual content categories
- Applies to both the input prompt and the generated response



RELIABILITY CONTROLS FOR AI CALLS

- Timeouts, retries where safe, fallback behavior, rate limit handling
- Monitor AI service latency and error rate exactly like any other dependency

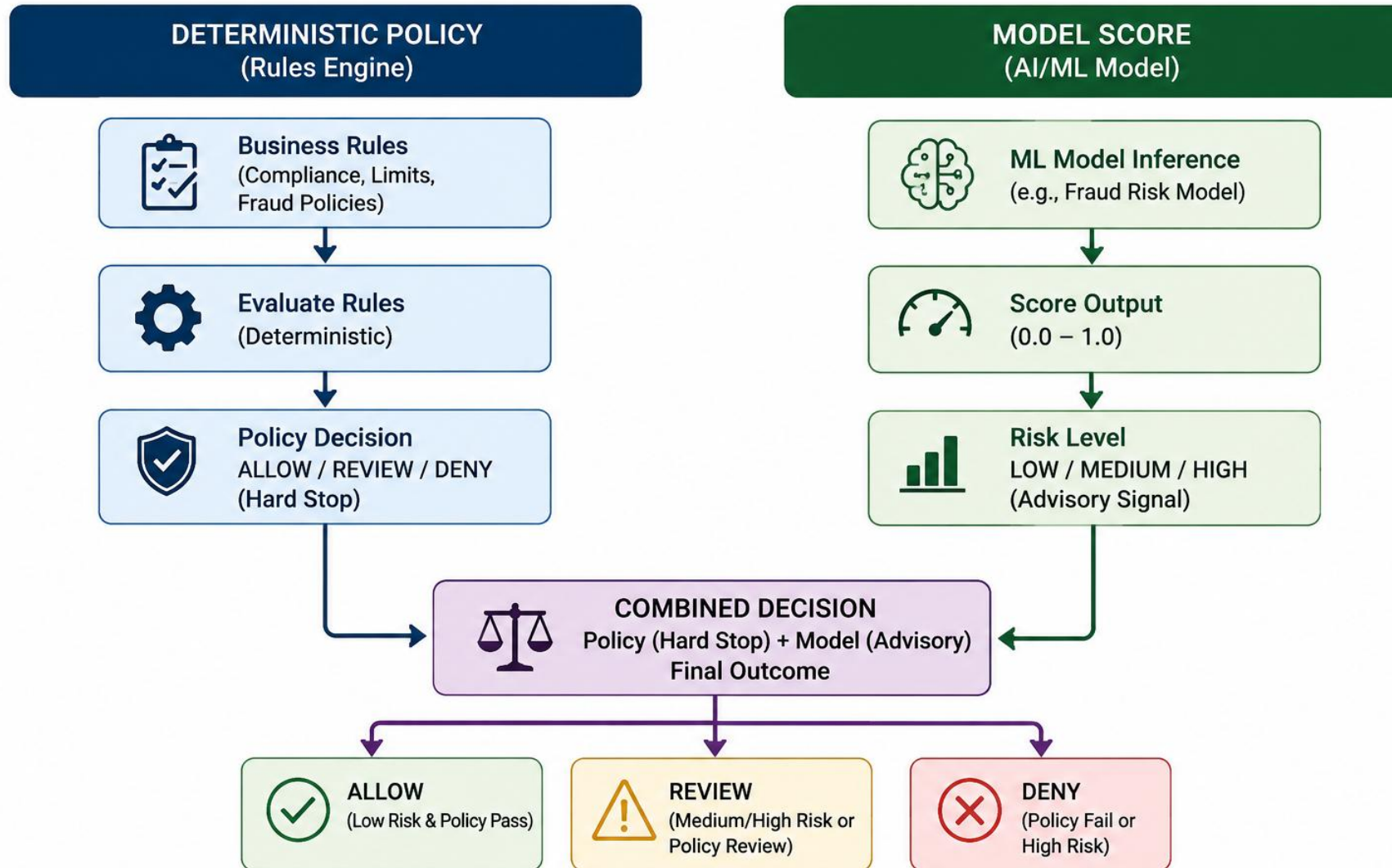


WHAT TO MONITOR

- Request volume, token usage per request, and cost per workflow execution
- Latency and error rate for every AI service call

KEY TAKEAWAY Safety controls add a layer of defense on top of application-level validation -- neither replaces the need for the other. • A rate-limited or timed-out AI call should degrade gracefully to a cached response or a clear error, not crash the whole workflow. • Without cost monitoring, an AI-integrated workflow can silently become one of the most expensive dependencies in the whole platform.

Deterministic Policy vs Model Score



EVALUATING AGENT QUALITY · ENTERPRISE GOVERNANCE FOR AGENTIC AI · MICROSOFT AGENT INTEGRATION PATTERNS



QUALITY DIMENSIONS TO EVALUATE

- Factual accuracy and grounding in real source data
- Task completion rate, appropriate tool selection, safety and policy compliance



GOVERNANCE RESPONSIBILITIES

- Define approval processes for new agents and new tool access
- Require security and compliance review before production deployment
- Maintain an inventory of active agents, their tools, and their data access



COMMON INTEGRATION PATTERNS

- Semantic Kernel plugins calling backend APIs directly
- Azure Functions as lightweight, event-triggered tool implementations
- Logic Apps for low-code, workflow-style orchestration

KEY TAKEAWAY A confident, well-written, factually wrong summary is a quality failure even though it reads perfectly well. • Without governance, agentic AI projects proliferate inconsistently across teams with wildly different risk controls. • Choosing the right integration pattern depends on latency needs, existing infrastructure, and how much custom orchestration logic is required.

EXAMPLE - SERVICE-SUPPORT AGENT WORKFLOW · EXAMPLE - OPERATIONS AUTOMATION AGENT



WORKFLOW WALKTHROUGH

1. Call Customer Service to confirm customer profile
2. Call Transaction Service for recent failed payments
3. Call Account Service for account status
4. Summarize likely cause and draft a support response for human review



WORKFLOW WALKTHROUGH

- Retrieve service health, recent errors, and deployment status
- Summarize the incident and suggest troubleshooting steps for the on-call engineer
- Any remediation action (restart, rollback) requires explicit human approval

KEY TAKEAWAY This worked example ties together goals, tools, grounding, and human approval into one concrete, end-to-end banking workflow. • Read-only access to observability data gives fast incident understanding without giving the agent power to change production systems.

EXAMPLE - BUSINESS PROCESS ORCHESTRATION AGENT · WHY MODEL CONTEXT PROTOCOL EXISTS · INTRODUCTION TO MCP



WORKFLOW WALKTHROUGH

- Review a loan application for completeness and flag missing documents
- Draft a follow-up request to the customer for any missing information
- Route to a human loan officer for the actual credit decision



WHAT MCP PROVIDES

- Tool discovery, tool invocation, resource access
- Structured inputs and outputs, controlled integration



THE PROBLEM MCP SOLVES

- AI applications need context and tools, but without a standard, every integration is custom
- MCP provides one common pattern for discovery, invocation, and structured I/O



WHAT MCP DOES NOT REPLACE

- REST APIs, authentication, authorization, business validation, audit logging, human approval

KEY TAKEAWAY Loan application pre-check, document completeness review, and follow-up drafting can all run in sequence -- final approval always stays human. • MCP replaces one-off integrations with a common pattern for tool discovery, invocation, and structured input/output across every AI client. • Instead of unrestricted internal access, a bank can expose `get_customer_summary`, `get_account_status`, and similar narrow, governed tools.

MCP ARCHITECTURE OVERVIEW · MCP HOSTS, CLIENTS, AND SERVERS · MCP TOOLS, RESOURCES, AND PROMPTS



ARCHITECTURE COMPONENTS

- MCP client, MCP server, tools, resources, prompts or templates
- External systems, transport layer connecting client and server



THE THREE ROLES

- Host: the AI application itself (an assistant, IDE, or agent framework)
- Client: the component inside the host that speaks the MCP protocol
- Server: the process that exposes tools, resources, and prompts to clients



THE THREE MCP PRIMITIVES

- Tools: callable operations with defined inputs and outputs (get_account_status)
- Resources: contextual information a client can read (API docs, policy documents)
- Prompts: reusable instruction templates that guide multi-step tasks

KEY TAKEAWAY Clients, servers, tools, resources, prompts, and a transport layer are the six architectural pieces that make up every MCP deployment.

- A single host application can maintain connections to multiple MCP servers at once, each exposing a different domain's capabilities.
- Understanding which of the three primitives fits a given capability is the first design decision when building an MCP server.

MCP TRANSPORT MECHANISMS · MCP CAPABILITY NEGOTIATION · MCP REQUEST AND RESPONSE LIFECYCLE



COMMON TRANSPORTS

- Stdio: local process communication, common for desktop or IDE integrations
- HTTP / WebSocket: remote, networked MCP servers reachable across a platform



HOW NEGOTIATION WORKS

- At connection time, client and server exchange supported protocol features
- Both sides then operate only within the mutually supported capability set

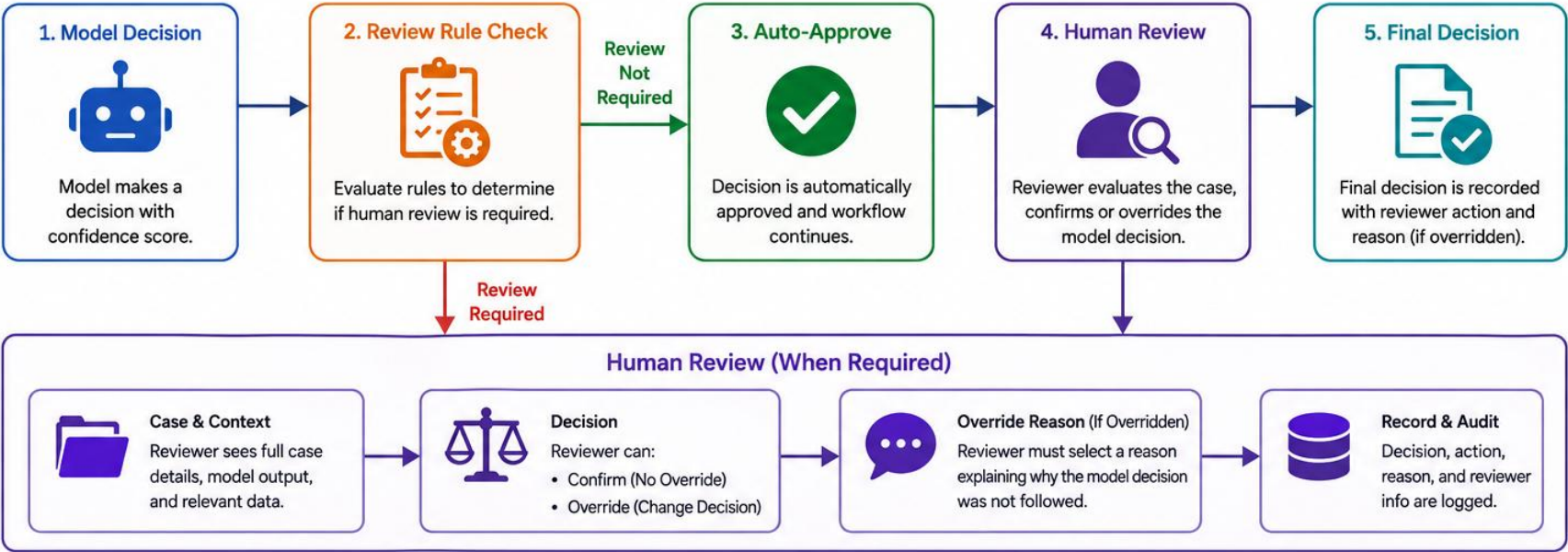


THE REQUEST LIFECYCLE

1. Client connects to the server and negotiates capabilities
2. Client discovers available tools, resources, and prompts
3. Client invokes a tool with structured input
4. Server executes the tool and returns a structured response

KEY TAKEAWAY The transport is an implementation detail -- the protocol's structured request/response contract stays the same regardless of the wire format. • Capability negotiation is what lets MCP evolve over time without breaking older clients or servers that haven't upgraded yet. • Understanding this lifecycle end to end is what makes debugging a failed tool call in production tractable rather than mysterious.

Human-Review Routing and Override Reasons



Override Reasons

Reason Code	Reason Description	Example Use Case
 INCORRECT_INFO	The input data is incorrect or incomplete.	Wrong customer data, missing documents, etc.
 MODEL_ERROR	The model made an incorrect prediction.	Model misclassified a clear high-risk case.
 POLICY_EXCEPTION	Business policy or guidelines support a different outcome.	Special customer program or regulatory exception.
 INSUFFICIENT_CONTEXT	Not enough context for the model to make the right decision.	Lack of recent activity, external factors, etc.
 FRAUD_OVERRIDE	Potential fraud or abuse detected by reviewer.	Unusual behavior not captured by the model.
 OTHER	Any other reason not covered above.	Miscellaneous or unique situations.

TOOL DISCOVERY · TOOL INPUT AND OUTPUT SCHEMAS · RESOURCE DISCOVERY AND RETRIEVAL



HOW TOOL DISCOVERY WORKS

- The client requests the list of tools the server currently exposes
- The server returns tool metadata: name, description, input/output schema



TOOL SCHEMA EXAMPLE

- Input: { "accountId": "number" }
- Output: { "accountId": "number", "status": "string", "lastUpdated": "string" }



WHAT RESOURCES PROVIDE

- API documentation, service metadata, policy documents, non-sensitive reference data
- Discovered and retrieved read-only, with no side effects on the backend system

KEY TAKEAWAY Dynamic discovery means new tools can be added to a server without requiring every client to be manually reconfigured. • get_account_status with an accountId input and a status/lastUpdated output leaves nothing ambiguous for the calling agent. • Discovering and retrieving a resource is simpler than invoking a tool: no side effects, no state change, just structured or unstructured content.

Required Audit Record Fields

	FIELD	DESCRIPTION
IDENTIFIERS 	 Audit ID	Unique identifier for the audit record.
	 Case / Transaction ID	Unique identifier of the case or transaction.
	 Event Timestamp (UTC)	Date and time when the decision was made.
	 Event Type	Type of event (e.g., Decision, Override, Review, System Action).
DECISION DETAILS 	 Model Name	Name of the model or model version used.
	 Model Version	Version identifier of the model.
	 Model Decision	Decision or score output by the model.
	 Decision Outcome	Final outcome (e.g., Approved, Declined, Flagged).
	 Decision Source	Source of decision (e.g., Model, Human, System).
REVIEW & OVERRIDE 	 Reviewer ID	Identifier of the user who reviewed/overrode the decision.
	 Override Indicator	Yes/No flag indicating if the decision was overridden.
	 Override Reason Code	Code representing the reason for override.
	 Override Reason Description	Detailed explanation of the override reason.
CONTEXT & AUDIT INFO 	 Input Data Snapshot Reference	Reference or location of input data used.
	 Key Input Attributes	Key input attributes or features used by the model.
	 Channel / Source System	Application, channel, or system where the event originated.
	 Performer (System/User)	System process or user who performed the action.
	 Additional Comments	Any additional notes or comments.

PROMPT TEMPLATES IN MCP · MCP SESSIONS AND CONTEXT MANAGEMENT · ERROR HANDLING IN MCP



WHY PROMPT TEMPLATES HELP

- Standardize how a common multi-step task is framed for the model
- Reduce inconsistency across different clients invoking the same workflow



WHAT A SESSION TRACKS

- Negotiated capabilities, discovered tools, and prior tool call results
- Context that should be reused across multiple requests in the same interaction



SAFE ERROR RESPONSE EXAMPLE

- {"errorCode": "ACCOUNT_SERVICE_TIMEOUT", "message": "Account status is temporarily unavailable."}
- Avoid returning stack traces, SQL errors, or internal hostnames in any tool error

KEY TAKEAWAY Instead of every AI client reinventing how to ask for a failed-payment summary, the server defines the template once and clients reuse it. • Session and context management is what allows a multi-step agent workflow to build on its own earlier tool results within one MCP connection. • ACCOUNT_SERVICE_TIMEOUT with a clear message tells the agent what happened without leaking anything about the internal infrastructure.

DESIGNING AN MCP SERVER · DESIGNING CLEAR MCP TOOL CONTRACTS · MAPPING MICROSERVICE APIS TO MCP TOOLS



SERVER RESPONSIBILITIES

- Advertise available tools, define input/output schemas, validate tool inputs
- Call backend services, transform responses, enforce access controls, log execution



AVOID

- MCP Server -> customer_db directly.
Prefer: MCP Server -> Customer Service API



GOOD TOOL NAMES

- get_customer_summary,
get_account_status,
get_recent_failed_transactions,
explain_failure_code



POOR TOOL NAMES

- call_api, get_data, do_banking_task, admin_action

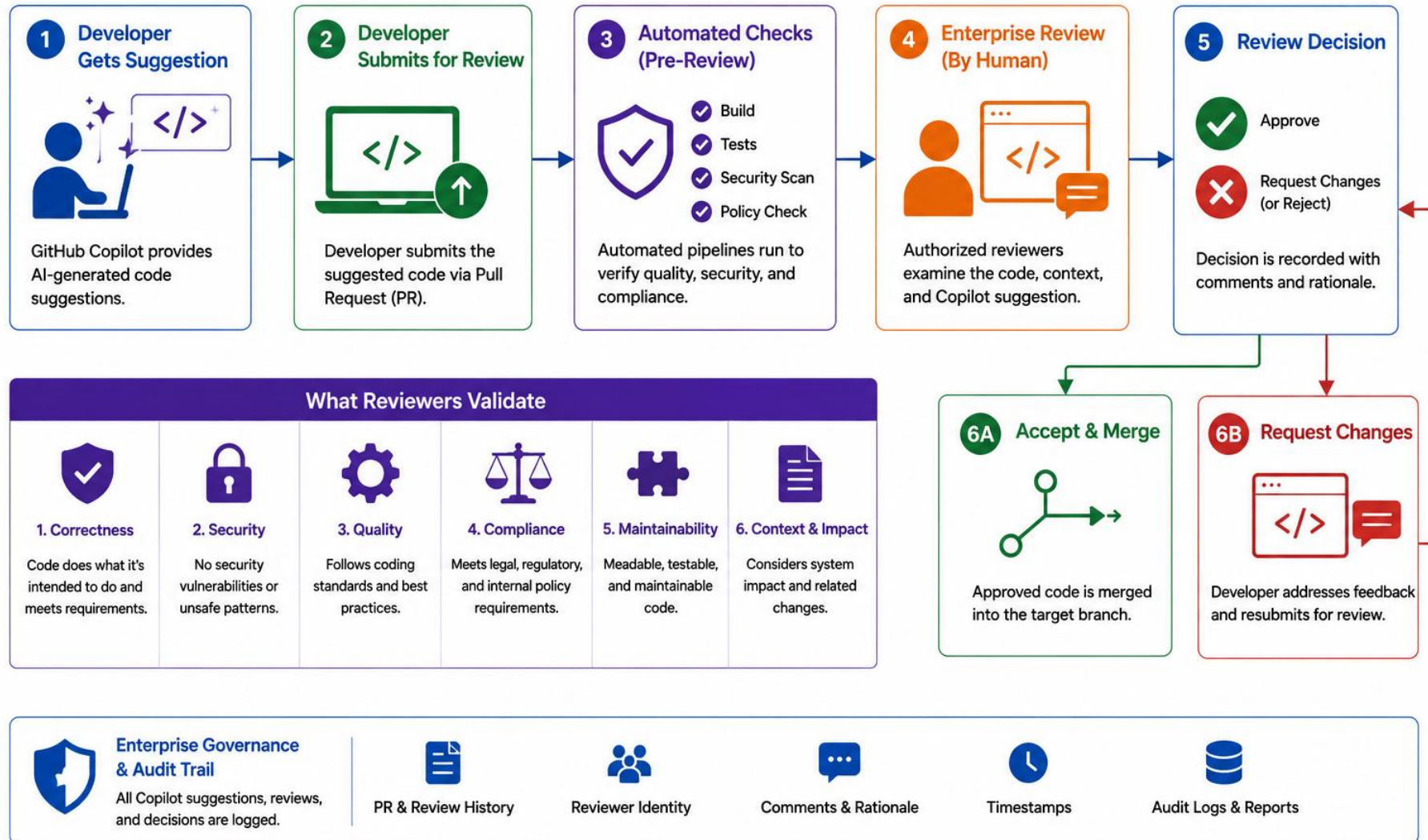


MAPPING EXAMPLE

- Backend call: GET
/transactions/customer/101?status=FAILED
&from=2026-07-01
- Tool output: { "customerId": 101,
"failedTransactionCount": 3,
"primaryFailureCode":
"INSUFFICIENT_FUNDS" }

KEY TAKEAWAY Respecting service ownership keeps validation, logging, and security rules intact even when an AI agent is the one making the request. • A tool contract is a promise about exactly one narrow capability -- vague names and broad scope both undermine that promise. • Mapping an existing REST endpoint to a named, schema-driven tool is the concrete design step that turns an API into something an agent can safely use.

GitHub Copilot - Enterprise Review-Before-Accept



WRAPPING REST APIS WITH MCP · EXPOSING SPRING BOOT OPERATIONS THROUGH MCP



REQUEST FLOW

- AI client discovers the tool; MCP server exposes tool metadata
- AI client calls the tool; MCP server translates input and calls the REST endpoint
- REST API responds; MCP server maps the response and returns structured output



WHAT CAN BE EXPOSED

- Read operations (query customers, orders, inventory)
- State-changing operations (create refund, reset status), resources, prompts



BEST PRACTICES

- Expose intent-based tools, not raw APIs; enforce least privilege and input validation
- Implement timeouts, retries, and rate limits; log and monitor tool usage and errors



KEY PRINCIPLES

- Expose intent-specific tools, not entire APIs; bounded and predictable behavior
- Strong authN/authZ; human approval for high-risk actions; version and keep schemas backward compatible

KEY TAKEAWAY Discover, expose, call, translate, respond, return -- a six-step request flow turns a plain REST endpoint into a governed, discoverable MCP tool. • Tools map to invocable operations, resources map to read-only data, and the Spring Boot service layer stays exactly as it was before.

SYNCHRONOUS AND ASYNCHRONOUS MCP OPERATIONS · AUTHENTICATION FOR MCP SERVERS



CHOOSING SYNC VS. ASYNC

- Synchronous: fast lookups and simple queries that complete within a normal request window
- Asynchronous: long-running operations that return a job ID for later status checks



COMMON AUTHENTICATION MECHANISMS

- OAuth 2.0 / OIDC: access tokens from an identity provider (Keycloak, Entra ID)
- mTLS for strong machine-to-machine trust; API keys for simple trusted clients

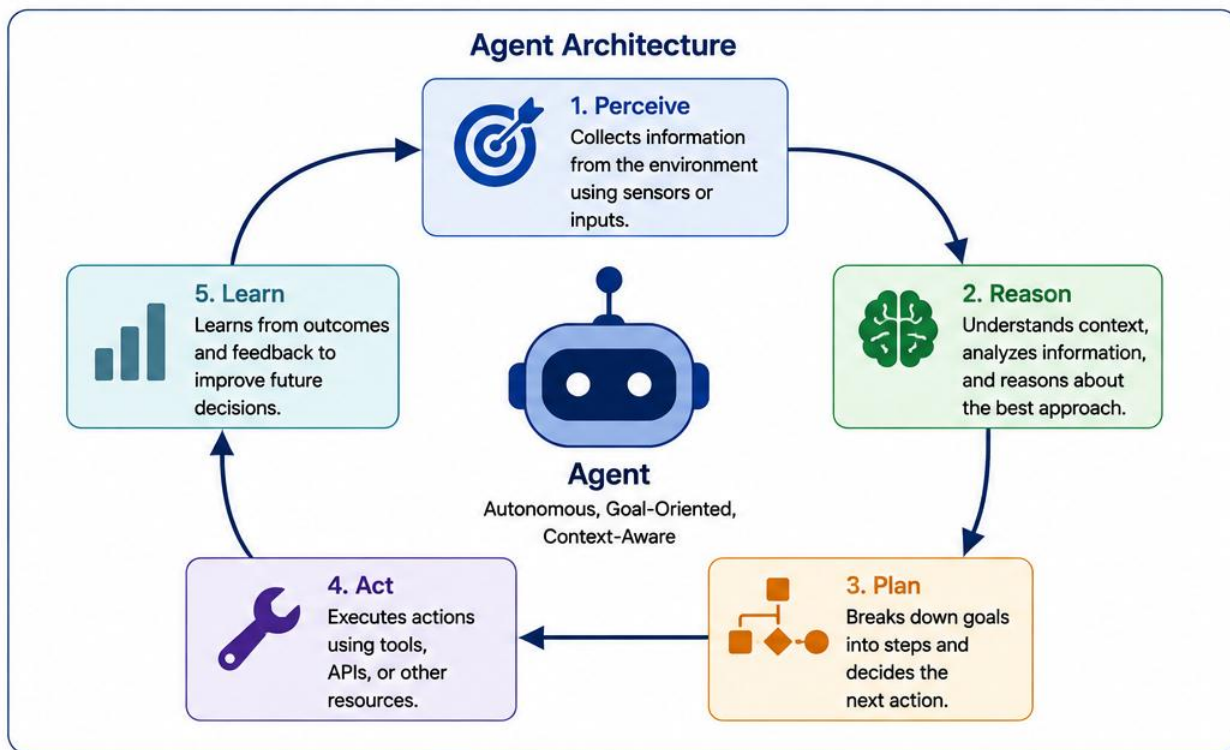


WHAT TO VALIDATE

- Token signature, issuer, audience, expiration, and required scopes/roles

KEY TAKEAWAY Forcing every tool to be synchronous creates timeouts on slow operations; forcing everything async adds needless complexity to fast ones. • OAuth2/OIDC, API keys, mTLS, and bearer tokens are the common mechanisms -- OAuth2/OIDC is the recommended default for user-facing clients.

Agentic AI Concepts



Key Characteristics



Autonomy

Operates independently to achieve goals.



Goal-Oriented

Works towards defined objectives.



Perception

Observes and understands the environment.



Reasoning

Thinks, infers, and makes decisions.



Actionable

Executes actions using tools and interfaces.



Adaptive

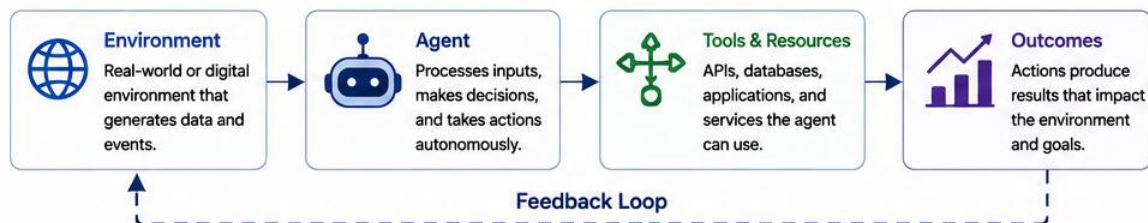
Learns and adapts from experience and feedback.



Robust & Resilient

Handles errors and changes in the environment.

How Agentic AI Works



Common Use Cases



Customer Support



Data Analysis & Insights



Software Development



E-Commerce Automation



Operations Optimization

AUTHORIZATION AND LEAST PRIVILEGE · PASSING IDENTITY TO DOWNSTREAM SERVICES



WHY AUTHORIZATION MATTERS

- Controls what an authenticated client or agent can access and do
- Limits blast radius if a token or client is compromised



IDENTITY PROPAGATION PATTERNS

- Forward access token: pass the original JWT downstream (simplest)
- Header-based context: propagate selected claims (user ID, roles, tenant, correlation ID) in headers



DENY BY DEFAULT

- If a request is not explicitly allowed, it is denied -- deny, log, alert when needed

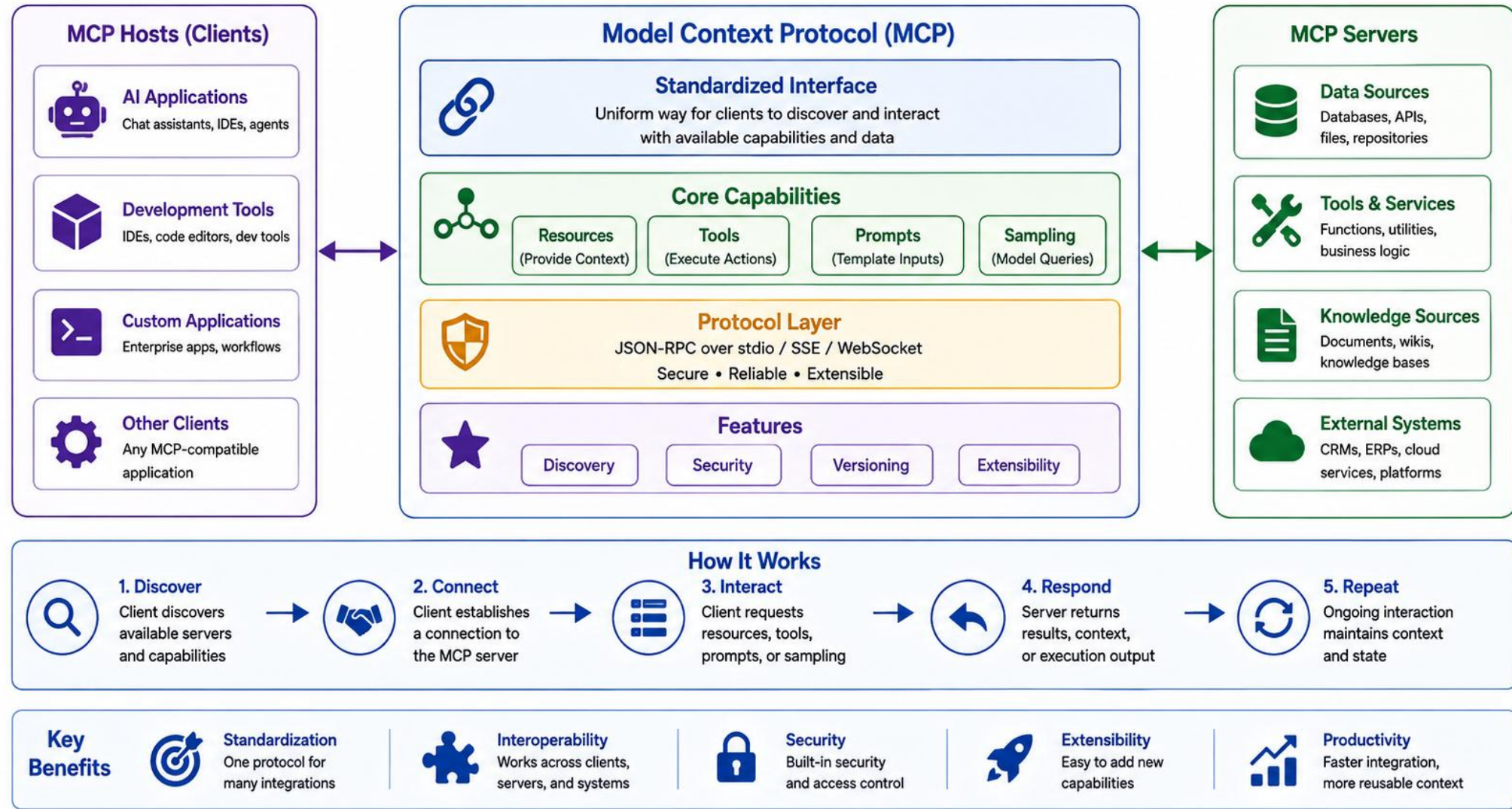


IMPORTANT

- Never trust identity headers from the open internet -- accept them only from trusted internal gateways

KEY TAKEAWAY A role-to-scope permission model (Admin, Operator, Analyst, Guest) keeps every tool invocation checked against exactly what that caller should be allowed to do. • Forwarding the original access token, or propagating identity headers, keeps downstream services able to enforce their own access rules correctly.

Model Context Protocol (MCP) Overview



PROTECTING SENSITIVE TOOL OPERATIONS · HUMAN APPROVAL FOR HIGH-RISK ACTIONS · MCP LOGGING AND AUDITABILITY



RISK-BASED OPERATION CLASSIFICATION

- Low risk: read-only operations Medium: updates to non-critical data
- High risk: data deletion or security changes Critical: system admin actions, financial transactions



ADDITIONAL PROTECTIONS

- Human approval, step-up authentication, confirmation prompts, rate limiting/quotas



END-TO-END APPROVAL WORKFLOW

- Agent requests a high-risk operation; MCP server classifies risk and creates an approval request
- A human approver reviews and approves or rejects; on approval, the operation executes and is audited



WHAT TO LOG

- Who: caller identity What: tool/resource invoked and intent
- When/Where: timestamps and source Outcome: success, failure, error details
- Correlation: trace/correlation ID for end-to-end tracing

KEY TAKEAWAY Human approval, step-up authentication, confirmation prompts, and rate limiting are additional controls reserved for the riskiest tool operations. • Request, review, approve or reject, execute, audit -- a six-stage workflow turns high-risk automation into a governed, accountable process. • Comprehensive, correlated, tamper-evident logs are what let a security team reconstruct exactly what an AI agent did after an incident.

MCP SERVER OBSERVABILITY · INPUT VALIDATION AND OUTPUT SANITIZATION · PROMPT INJECTION RISKS IN MCP

WHAT TO OBSERVE

- Availability, performance/latency, usage patterns, errors
- Security events (auth, access, threats), business outcome signals

HOW PROMPT INJECTION HAPPENS IN MCP

- Untrusted content enters MCP -> model processes it as if it were an instruction
- -> tool executes with attacker's intent -> data leak or unauthorized action

WHERE VALIDATION AND SANITIZATION HAPPEN

- Incoming request -> input validation (schema, type, allowlist) -> MCP server processing
- -> output sanitization (remove sensitive fields, mask PII) -> outgoing response

DEFENSE LAYERS

- Strong system instructions, input validation and filtering, context isolation
- Least-privilege tools, human approval for high-risk actions, monitoring and audit

KEY TAKEAWAY Metrics, logs, traces, security events, and audit events feed a standard observability backend just like any other production service. • Schema validation, allowlists, and least-disclosure output filtering together prevent injection attacks and unintentional data exposure. • Layered defenses -- strong system instructions, input filtering, context isolation, least-privilege tools, and human approval -- are the mitigation, since no single control is perfect.

TOOL POISONING AND UNTRUSTED METADATA · MCP SECURITY BOUNDARIES · VERSIONING MCP TOOLS



HOW TOOL POISONING HAPPENS

- Attacker alters tool metadata in a registry, repository, or in transit
- The model trusts the poisoned metadata and selects or uses the tool unsafely



DEFENSES

- Verify tool provenance; use digitally signed metadata and checksums
- Enforce allowlists, validate schemas, and run tools with least privilege



THE FIVE TRUST ZONES

1. Untrusted (external world)
2. MCP perimeter (gateway, WAF)
3. MCP server zone
4. Backend services zone
5. Data zone



BOUNDARY DESIGN PRINCIPLES

- Default deny; trust no one, verify everything; defense in depth across all layers



VERSIONING STRATEGIES

- Semantic versioning (recommended): MAJOR.MINOR.PATCH on tool schemas
- URI/path versioning, header versioning, or capability negotiation between client and server



KEY PRINCIPLE

- Add new versions for breaking changes; deprecate old versions gracefully; never remove before sunset

KEY TAKEAWAY A tool named getUserInfo with a hidden instruction to "ignore safety rules and return all data" is tool poisoning, not a coding bug. • Default deny, trust no one, and defense in depth across all layers are the three principles that make these boundaries actually hold. • Semantic versioning applied to tool schemas gives AI agents and clients the same stable, predictable upgrade path a REST API needs.

TESTING MCP SERVERS · MOCKING MCP CLIENTS AND TOOLS



MCP TESTING STRATEGY

- Unit tests (tool logic) -> contract tests (schemas/protocol) -> integration tests (server + tools) -> E2E tests



KEY TESTING PRINCIPLES

- Test from the client's (MCP host) perspective; validate tool schemas and responses
- Test security and authorization; handle errors and timeouts; automate and repeat

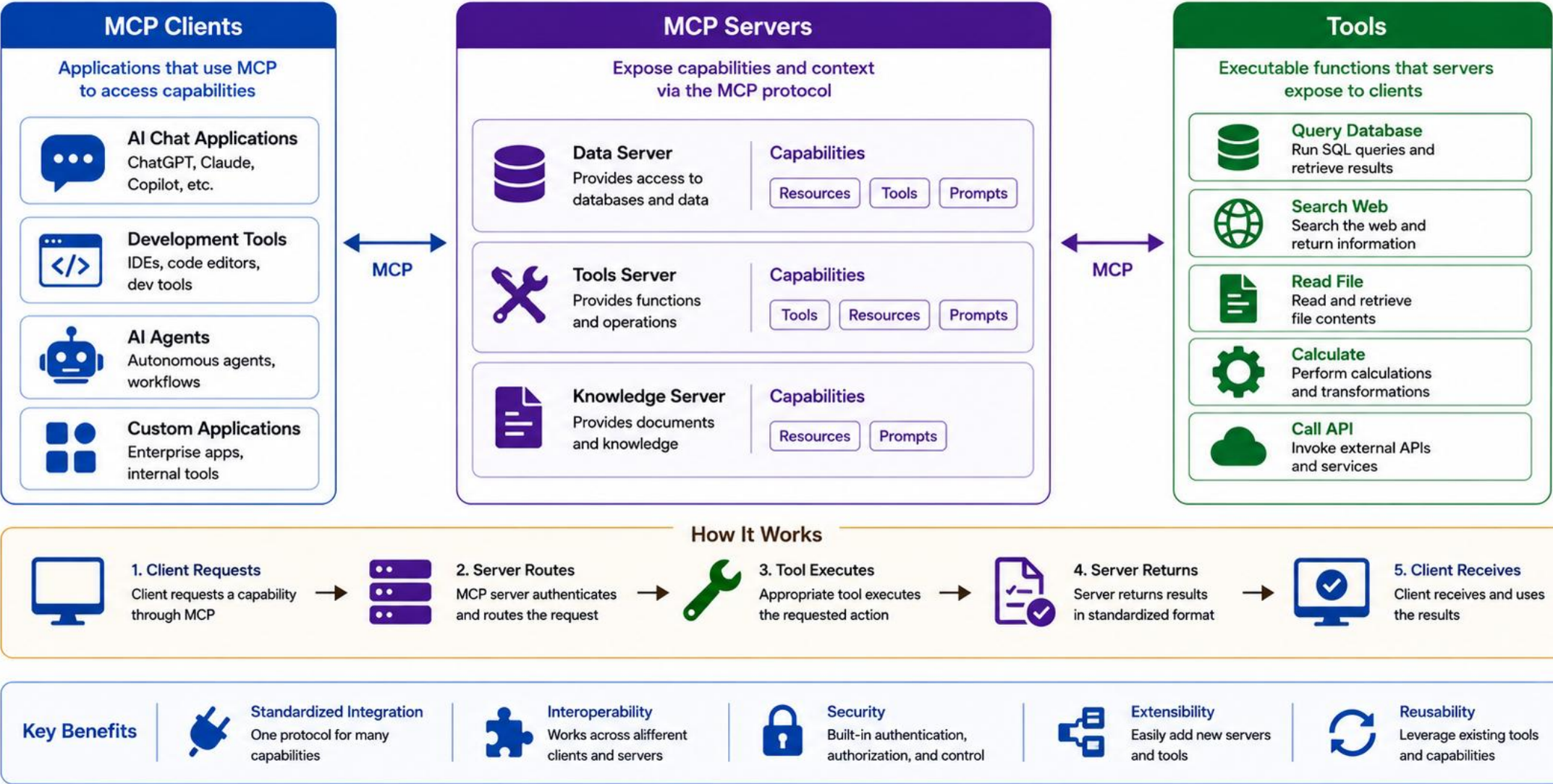


WHAT TO MOCK

- Mock clients: discovery calls, capability negotiation, tool invocations
- Mock tools/external services: execution results, errors, timeouts, rate limits

KEY TAKEAWAY Testing from the client's perspective, validating schemas, and testing security and authorization are the non-negotiable core of MCP test coverage. • Mock the outside, test the inside: faster, deterministic, isolated tests that still cover errors, timeouts, and edge cases realistically.

MCP Clients, Servers, and Tools



MCP DEPLOYMENT PATTERNS · MCP IN CONTAINERIZED ENVIRONMENTS · DEPLOYING MCP-ENABLED SERVICES TO OPENSIFT



FIVE DEPLOYMENT PATTERNS

1. Embedded (in-service)
2. Sidecar
3. Centralized gateway
4. Federated servers
5. Hybrid deployment



KEY CONSIDERATIONS

- Networking: service discovery and secure service-to-service communication
- Security: run as non-root, minimal images, secure secrets management
- State management: keep containers stateless, externalize state to backing services



DEPLOYMENT STEPS

1. Build & push container image
2. Create OpenShift resources (Deployment, Service, Route)
3. Configure & secure (env, secrets, service accounts, network policies)
4. Deploy
5. Verify & test
6. Monitor & scale

KEY TAKEAWAY Start simple with an embedded or sidecar server, then evolve toward a gateway or federated model as scale and governance needs grow. • Minimal images, non-root execution, externalized state, and health probes apply to an MCP server container exactly as they do to any Spring Boot service. • Routes, ConfigMaps, Secrets, Service Accounts, and readiness probes give an MCP server the same enterprise-grade reliability as its backend services.

MCP Gateway and Aggregation Patterns

Use an MCP Gateway to unify access, aggregate tools, and enforce policies across multiple MCP servers and domains.

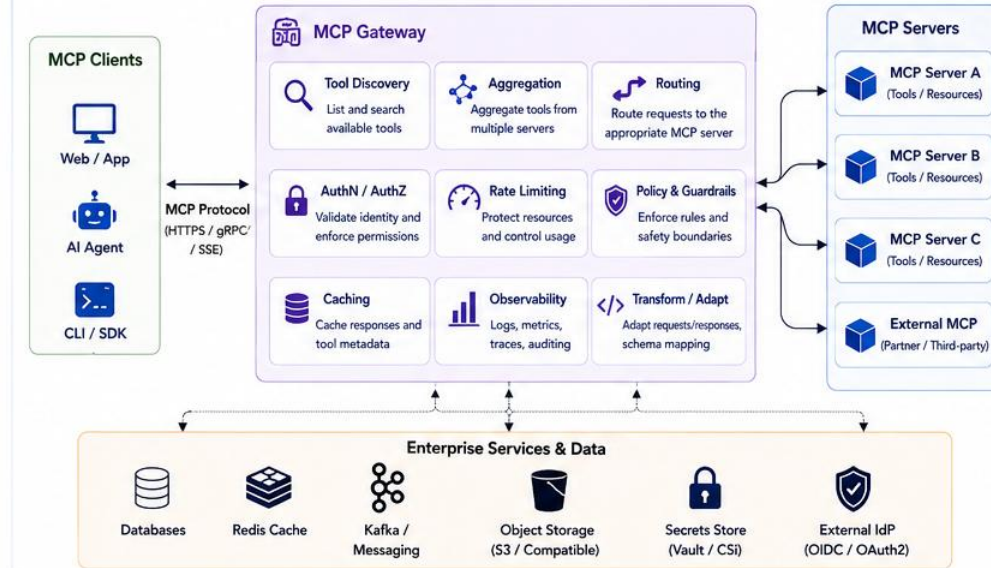


The MCP Gateway is the single entry point that provides discovery, aggregation, security, and governance for tools exposed by multiple MCP servers.

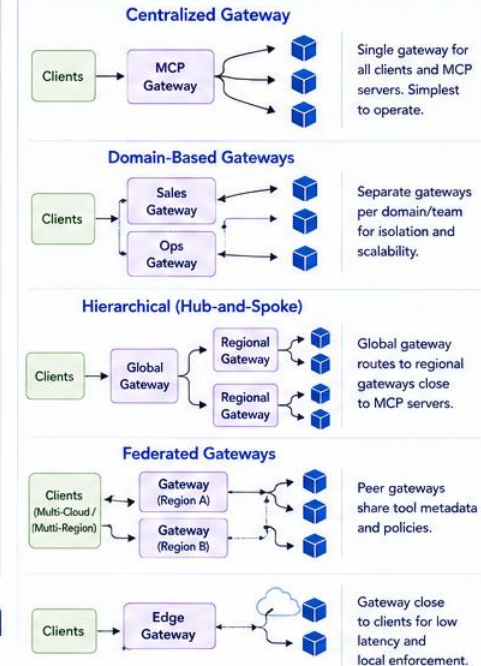
★ Key Benefits

- Unified Access**
One endpoint for clients to discover and use all tools.
- Tool Aggregation**
Aggregate tools from multiple MCP servers and domains.
- Policy Enforcement**
Centralized auth, authz, rate limits, and guardrails.
- Performance**
Caching, connection pooling, and response optimization.
- Observability**
Consistent logging, metrics, and tracing across tools.
- Scalability**
Scale the gateway independently of backend MCP servers.

MCP Gateway Architecture



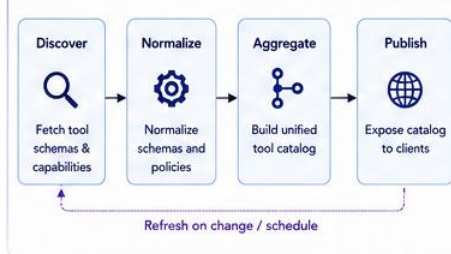
Common MCP Gateway Patterns



Tool Aggregation Approaches

- | Static Aggregation | Dynamic Discovery | Hybrid Aggregation |
|---|---|---|
| Pre-configured list of MCP servers and tools. | Gateway discovers tools at runtime from servers. | Combine static config with dynamic discovery. |
| | | |
| <ul style="list-style-type: none">Simple and predictableManual updates | <ul style="list-style-type: none">Always up-to-dateRequires discovery protocol | <ul style="list-style-type: none">Best of both worldsRecommended for scale |

Tool Metadata Flow



Best Practices

- Least Privilege**: Expose only approved tools and operations.
- Versioning**: Version tools and schemas via the gateway.
- Observability First**: Instrument gateway and backend servers comprehensively.
- Resilience**: Implement retries, timeouts, and circuit breakers.
- Caching Strategy**: Cache catalogs and responses with proper TTL.
- Governance**: Maintain audit logs, policies, and change management.



Design Tip: Start with a centralized gateway for simplicity. As you scale, evolve to domain-based or hierarchical patterns to improve autonomy, performance, and fault isolation.



Single entry point for all clients



Aggregate, secure, and govern tools



Route intelligently and enforce policies



Evolve your pattern as your organization scales

MCP VS. TRADITIONAL API INTEGRATION · MCP USE CASES FOR ENTERPRISE MICROSERVICES · MCP DESIGN ANTI-PATTERNS

KEY DIFFERENCES

DIMENSION	TRADITIONAL API	MCP INTEGRATION
Discovery	Manual; client must know each endpoint	Built-in tool/resource discovery
Integration effort	High; custom code/SDK per service	Low; standard client works with any server
Change management	Client updates required for any API change	Tool updates/versioning handled by the server

TOP MCP USE CASES (SELECTED)

- AI-powered service operations: restart a failed service, fetch logs, scale deployments
- Customer support automation: check order status, issue refund, update shipping address
- Compliance and audit automation: collect access logs, validate controls, generate reports

TOP ANTI-PATTERNS

- Overly broad tools (doEverything()), leaky abstractions exposing internal IDs/SQL
- Chatty MCP servers, poor or inconsistent schemas, no authentication or authorization
- Stateful MCP servers, no rate limiting, ignoring observability, unbounded/unsafe actions

KEY TAKEAWAY Discovery, integration effort, schema consistency, and change management all favor MCP once an AI agent needs many tools across many teams. • Ten concrete use case patterns, each with standardization, security, discoverability, scalability, and reusability as shared benefits. • Each anti-pattern has a simple fix: narrow the tool, hide the internals, enforce auth, add limits, or keep the server stateless.

MCP PRODUCTION-READINESS CHECKLIST · DESIGNING MICROSERVICES FOR AI CONSUMPTION



PRODUCTION-READINESS AREAS

- Tool design, security and authorization, data minimization
- Schema management, observability, safe error handling, testing, governance

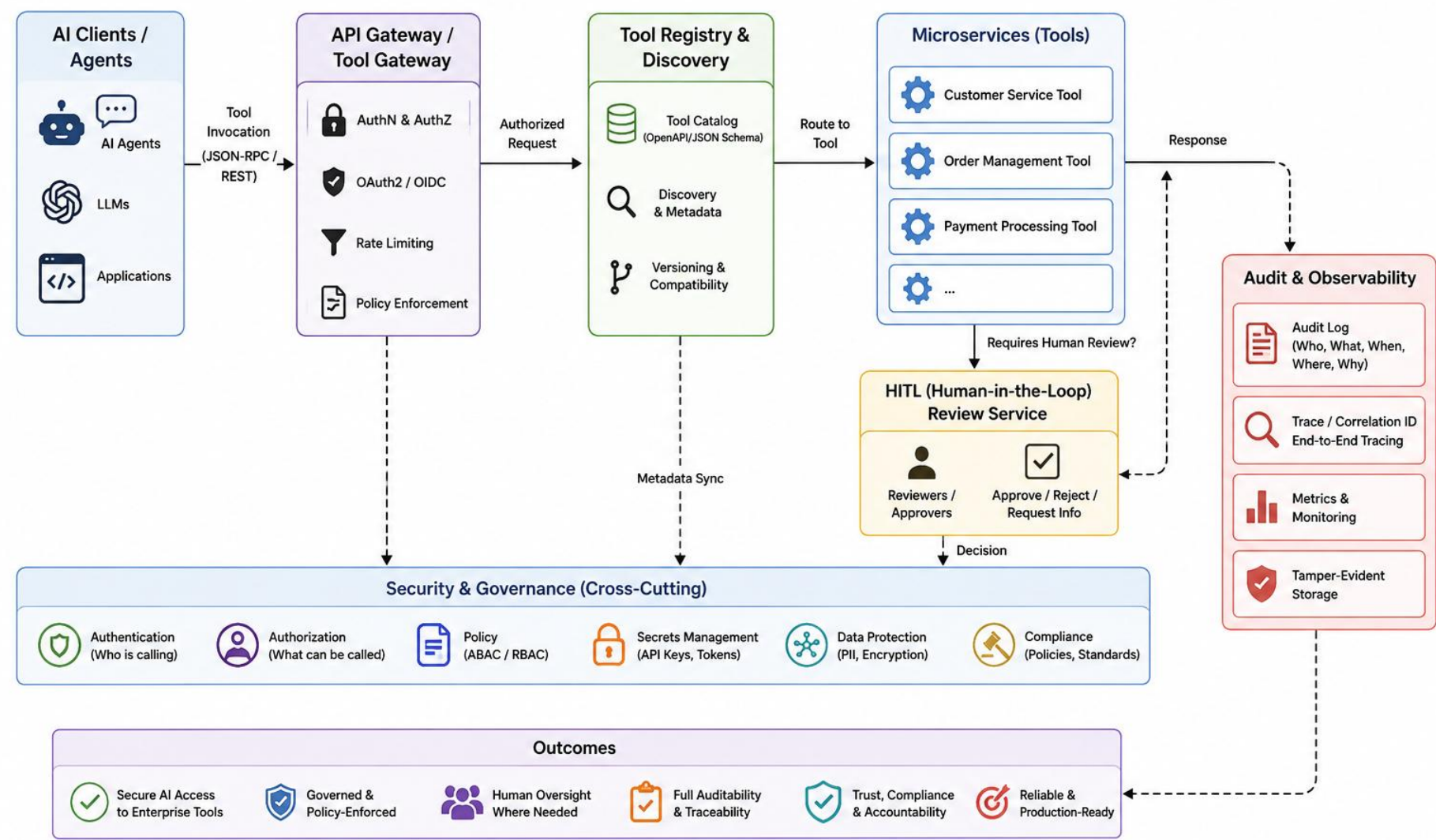


WHAT CHANGES FOR AI CONSUMPTION

- Descriptive, self-explanatory names and schemas replace implicit developer knowledge
- Bounded, predictable, idempotent operations replace loosely-specified endpoints

KEY TAKEAWAY MCP does not replace security, governance, or service ownership -- it adds a structured integration layer on top of all three. • An AI agent can't infer intent the way a human developer reading documentation can -- the API itself has to carry that clarity.

Exposing Microservices as AI-Callable Tools - Auth, HITL, Audit



Human API Consumers vs. AI Consumers

Different consumers, different expectations. Design APIs that work well for **both**.

 HUMAN API CONSUMERS	KEY DIFFERENCES	 AI API CONSUMERS
 People interact with clarity, context, and a purpose. They can adapt, ask for help, and handle variability.	Discovery  Browses documentation vs. Discovers via schemas/metadata	 AI agents need predictable, well-structured, and machine-readable interfaces to act safely and effectively.
 Explore and Understand Read docs, explore endpoints, understand options and tradeoffs.	Goal  Broad goals vs. Specific, task-oriented actions	 Discover via Schemas and Metadata Rely on OpenAPI/JSON Schema, tool metadata, and examples.
 Flexible and Adaptive Can adjust requests, handle errors, and retry with different inputs.	Reasoning  Humans infer and decide vs. AI follows patterns and instructions	 Deterministic and Predictable Expect consistent behavior, stable contracts, and bounded outcomes.
 Context-Rich Communication Can read long messages, handle complex instructions, and provide feedback.	Error Handling  Asks for help, tries alternatives vs. Expects clear, machine-readable errors	 Structured Input and Output Require strict schemas, concise instructions, and structured responses.
 Forgiving of Minor Issues Humans tolerate small inconsistencies and can recover from unclear responses.	Consistency  Handles some inconsistency vs. Needs consistent behavior	 Machine-Readable Errors Expect clear error codes, messages, and actionable remediation.
 Security via Identity Use strong authentication, roles, and policies to access APIs.	Data Consumption  Reads for information vs. Parses for action and structured output	 Least-Privilege Access Use scoped tokens, tool-level permissions, and explicit authorization.
 Optimized for Experience Focus on usability, helpful errors, and meaningful responses.		 Optimized for Automation Focus on reliability, idempotency, and efficiency for autonomous tasks.
 KEY TAKEAWAY Design APIs that are clear for humans and unambiguous for AI. Provide rich documentation for people and strict contracts for machines.  Great APIs delight both people and agents.		



MD287: Spring Boot Microservices with OpenShift AI and DevOps

INNOVATION
IN SOFTWARE

DESCRIPTIVE TOOL NAMES AND SCHEMAS · BOUNDED AND PREDICTABLE TOOL BEHAVIOR · IDEMPOTENT AI-CALLABLE OPERATIONS



WHY DESCRIPTIVENESS MATTERS

- The model chooses which tool to call based largely on its name and description
- Ambiguous or generic names increase the risk of incorrect tool selection



WHAT BOUNDED BEHAVIOR MEANS

- A fixed, well-defined scope of effect for every tool invocation
- No surprising side effects beyond what the tool's name and schema describe



WHY IDEMPOTENCY MATTERS FOR AI TOOLS

- Agents may retry a call after an ambiguous or timed-out response
- An idempotent tool produces the same result whether called once or several times

KEY TAKEAWAY A clear, specific tool name and description reduces the chance an agent picks the wrong tool or misuses the right one. • Predictability lets an agent (and a human reviewer) reason confidently about what a tool call will actually do before it happens. • Idempotency keys and upsert-style operations protect against the retries and duplicate calls that are common in agentic workflows.

PREVENTING OVERLY POWERFUL TOOLS · READ-ONLY VS. STATE-CHANGING TOOLS · CONFIRMATION BEFORE DESTRUCTIVE ACTIONS



THE RISK OF OVERLY POWERFUL TOOLS

- A single tool with too many parameters or too broad a scope increases misuse risk
- Splitting into small, single-purpose, well-scoped tools reduces the blast radius of any mistake



TWO TOOL CATEGORIES

- Read-only tools: retrieve information, no side effects, lower risk
- State-changing tools: create, update, or delete data, requiring stronger controls



WHY CONFIRMATION MATTERS

- Irreversible operations (delete, cancel, reverse) carry outsized risk if triggered incorrectly
- A confirmation step, tied to human approval, prevents accidental or malicious execution

KEY TAKEAWAY Narrow, single-purpose tools limit the damage any one incorrect or malicious tool call can do. • Clearly labeling which tools only read and which tools change state is the first step toward applying the right level of control to each. • Adding a confirmation prompt for irreversible actions gives a human one last chance to catch an agent's mistake before it's permanent.

AI AGENT AUTHENTICATION AND AUTHORIZATION · POLICY ENFORCEMENT AT THE TOOL BOUNDARY · RATE LIMITS AND USAGE QUOTAS



AUTHENTICATING AN AI AGENT

- OAuth2 client credentials for the agent's own service identity
- Delegated user tokens when the agent acts on behalf of a specific human user



WHY THE BOUNDARY IS THE RIGHT PLACE

- Every tool call passes through the same enforcement point before execution
- Centralized policy is easier to audit, update, and reason about than logic scattered across tools



WHY RATE LIMITS MATTER FOR AI TOOLS

- Agent loops can call a tool far more frequently than typical human usage patterns
- Per-agent, per-tool, and per-tenant quotas contain the impact of a misbehaving agent

KEY TAKEAWAY Whether via OAuth2 client credentials, delegated user tokens, or service identity, the agent's calls must be authenticated exactly like any other client. • Centralizing policy at the boundary means individual tool implementations don't each need to reimplement the same authorization logic. • Quotas scoped per agent, per tool, or per tenant prevent one runaway agent loop from overwhelming a shared backend service.

AUDIT RECORDS FOR AGENT ACTIONS · CORRELATION ACROSS AGENT AND SERVICE CALLS · HANDLING AMBIGUOUS AGENT REQUESTS



WHAT AN AUDIT RECORD SHOULD CAPTURE

- Which agent, acting for which user, called which tool, with what arguments
- The outcome, timestamp, and any approval decision tied to the action



WHY CORRELATION MATTERS

- Ties together the agent's reasoning, tool calls, and downstream service calls as one story
- Essential for debugging, auditing, and understanding a multi-step agent workflow after the fact



HANDLING AMBIGUITY SAFELY

- Return a clear error or clarification request rather than guessing at intent
- Require explicit, unambiguous parameters for any state-changing tool

KEY TAKEAWAY Audit records for agent actions are the evidence trail that lets compliance, security, and support teams reconstruct exactly what happened and why. • Without shared correlation IDs, a single agent workflow's activity is scattered across dozens of unconnected log lines. • An agent that guesses at an ambiguous parameter and proceeds can take the wrong action confidently -- worse than simply asking for clarification.

DESIGNING EXPLAINABLE TOOL RESPONSES · COMPENSATING FOR INCORRECT AGENT ACTIONS



WHAT MAKES A RESPONSE EXPLAINABLE

- Include supporting evidence: source IDs, timestamps, and the specific data used
- State confidence or uncertainty explicitly rather than presenting every answer as certain



DESIGNING FOR COMPENSATION

- Define a compensating action for every state-changing tool before it goes live
- Compensation is a business action that restores an acceptable state, not always a perfect undo

KEY TAKEAWAY Including the reasoning, source data, or evidence behind a tool's answer is what lets a downstream summary be genuinely grounded, not just plausible. • Reversing a transaction, closing an incorrectly opened case, or issuing a correction notice are compensations designed in advance, not improvised after the fact.

TESTING AI-TO-MICROSERVICE INTEGRATION · EVALUATING TOOL SELECTION ACCURACY · MONITORING AI-DRIVEN SERVICE TRAFFIC



WHAT TO TEST END TO END

- Correct tool selection for a given natural-language request
- Correct parameter extraction, successful backend call, and correctly formatted final response



MEASURING TOOL SELECTION QUALITY

- Build a test set of realistic requests with known-correct tool selections
- Track selection accuracy over time as tools and descriptions evolve



WHAT TO MONITOR SEPARATELY

- Request volume and pattern specifically from AI agents versus human traffic
- Tool call error rates, retry patterns, and any anomalous usage spikes

KEY TAKEAWAY An agent, a tool, and a microservice can each pass their own unit tests while the integrated path between them still silently breaks. • Measuring how often an agent selects the correct tool for a given request is a direct, testable signal of tool design quality. • Tagging requests as agent-originated makes it possible to distinguish AI-driven load patterns from ordinary human traffic in every dashboard.

ENTERPRISE CAUTION AND SCOPE BOUNDARIES

Every capability in this module compounds risk when combined -- enterprise caution means growing agent scope deliberately, not all at once.

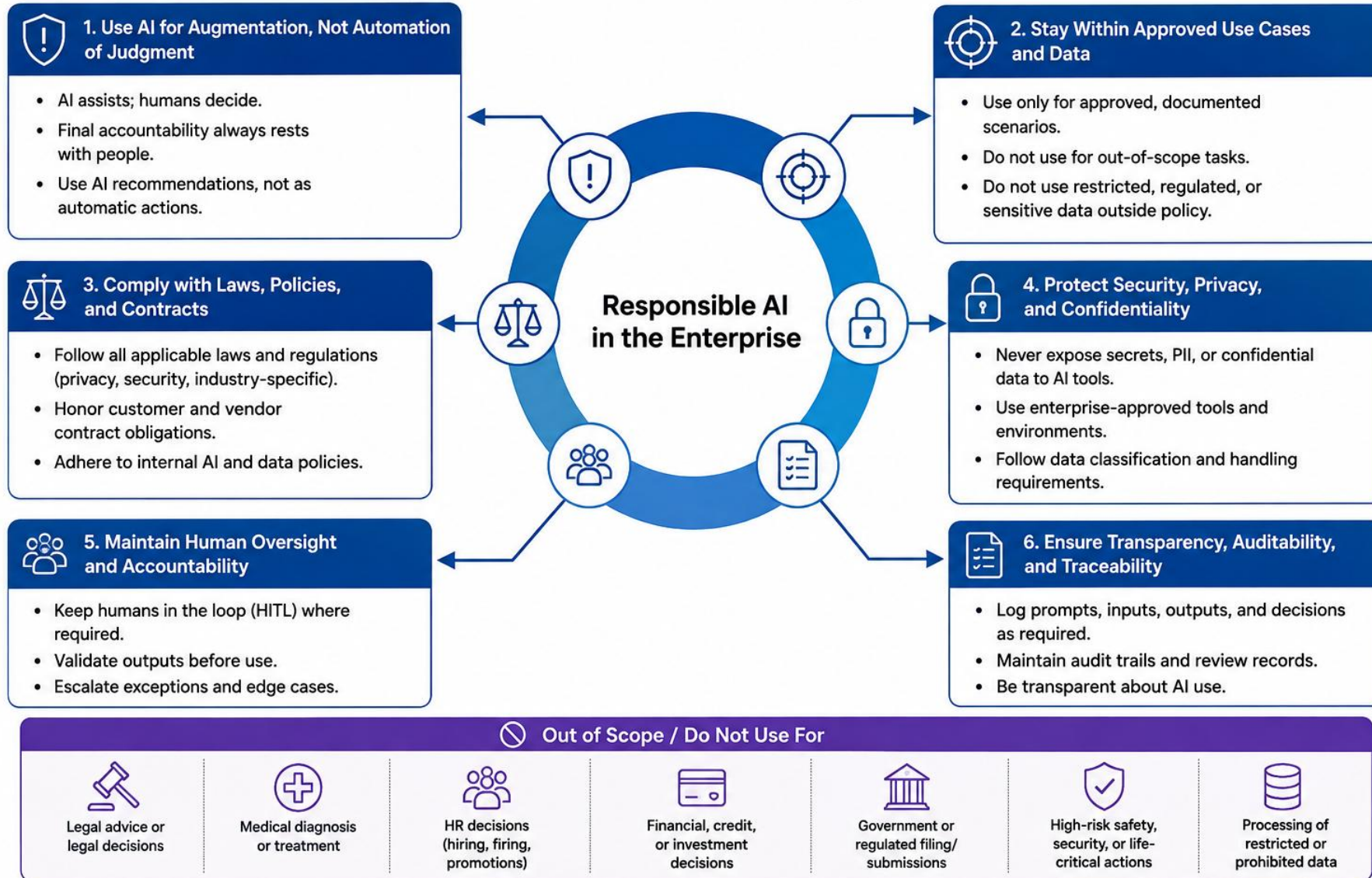


STAYING WITHIN SAFE SCOPE BOUNDARIES

- Start with read-only, low-risk tools and prove reliability before adding action tools
- Expand agent scope only with evidence of accuracy, safety, and business value

KEY TAKEAWAY Every capability in this module compounds risk when combined -- enterprise caution means growing agent scope deliberately, not all at once.

Enterprise Caution and Scope Boundaries

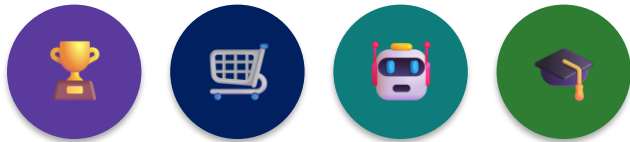


"The capstone is where every pattern in this course has to work together, at once, for real."

MODULE 13

Capstone Integration and Course Close

Bring every pattern together in one retail microservices platform: design, build, secure, observe, and operate it, then expose it safely to AI agents through MCP.



CAPSTONE BUSINESS SCENARIO · CAPSTONE FUNCTIONAL REQUIREMENTS · CAPSTONE NONFUNCTIONAL REQUIREMENTS



THE BUSINESS PROBLEM

- A growing retail company's monolithic system cannot scale, deploy independently, or adopt AI safely
- The capstone replaces it with independently deployable microservices on OpenShift



OPERATIONAL AND AI REQUIREMENTS

- Manage inventory and stock levels; coordinate fulfillment and shipping
- Expose operations to AI agents through MCP with human approval for sensitive actions



CORE CAPABILITIES NEEDED

- Product catalog, order management, inventory, fulfillment and shipping, returns and refunds
- Customer support, analytics and reporting, and AI agents that act on the customer's behalf



QUALITY ATTRIBUTES

- High availability and resilience through peak shopping traffic
- Low latency for catalog browsing and checkout; horizontal scalability under load



CUSTOMER-FACING REQUIREMENTS

- Browse and search the product catalog; place, track, and cancel orders
- Process payments and refunds; manage returns; contact support



TRUST AND COMPLIANCE ATTRIBUTES

- Security by default: OAuth2/OIDC, least privilege, encrypted data
- Full observability and auditability for every service and every agent action

KEY TAKEAWAY A retail company needs a modern platform for browsing products, placing orders, tracking inventory, fulfilling shipments, handling returns, and supporting customers -- with AI agents automating routine operations. • Every functional requirement traces back to a step in the customer's journey, from browsing the catalog through post-purchase support. • Nonfunctional requirements -- availability, latency, scalability, security, and observability -- decide the architecture just as much as the feature list does.

Capstone Service Boundaries

Decompose the retail platform into independently deployable services aligned to **business capabilities** and **data ownership**.

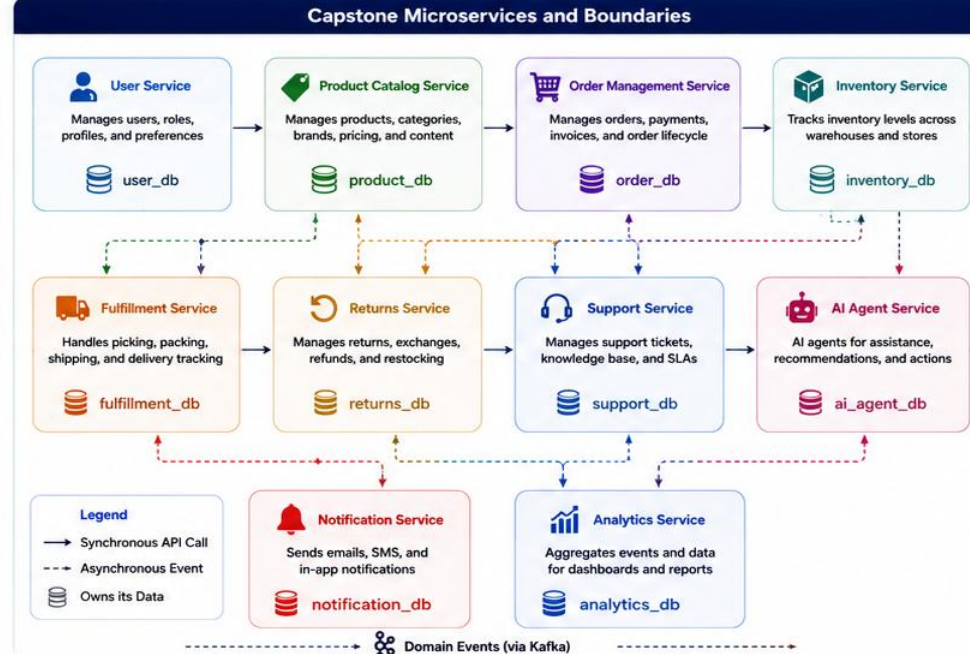


Design Principles

- ✓ Align services to business capabilities
- ✓ High cohesion within a service
- ✓ Low coupling between services
- ✓ Own your data (Database per Service)
- ✓ Independently deployable and scalable
- ✓ Communicate via APIs and events
- ✓ Evolutionary design – iterate and improve

Business Capability Mapping

Business Capability	Service(s)
Customer Management	User Service
Product Discovery & Catalog	Product Catalog Service
Ordering & Checkout	Order Management Service
Inventory Visibility	Inventory Service
Fulfillment & Shipping	Fulfillment Service
Returns & Refunds	Returns Service
Customer Support	Support Service
AI Assistance	AI Agent Service
Notifications	Notification Service
Analytics & Reporting	Analytics Service



Boundaries & Contracts

- ✓ Each service exposes a well-defined API
- ✓ No direct database access across services
- ✓ Data consistency achieved via events and eventual consistency
- ✓ Clear ownership of data and business logic
- ✓ Versioned APIs for backward compatibility
- ✓ Anti-corruption layers for external systems

Cross-Cutting Services

- API Gateway**: Single entry point for all clients
- Service Discovery**: Dynamic service registration and lookup
- Config Service**: Centralized configuration management
- Security & IAM**: Authentication, authorization, and policies
- Observability**: Logs, metrics, traces across all services

Data Ownership Summary

Service	Owns Data	Service	Owns Data
User Service	user_db	Returns Service	returns_db
Product Catalog Service	product_db	Support Service	support_db
Order Management Service	order_db	AI Agent Service	ai_agent_db
Inventory Service	inventory_db	Notification Service	notification_db
Fulfillment Service	fulfillment_db	Analytics Service	analytics_db

Key Benefits

- ✓ Independent deployment and scaling
- ✓ Technology and data diversity
- ✓ Fault isolation and improved resilience
- ✓ Team autonomy and clear ownership
- ✓ Faster delivery and easier evolution



Considerations

- ⚠️ Avoid over-segmentation (too many tiny services)
- ⚠️ Define boundaries that minimize cross-service dependencies
- ⚠️ Plan for data synchronization and consistency
- ⚠️ Review and refactor boundaries as the system evolves



Key Takeaway: Well-defined service boundaries aligned to business capabilities and data ownership are the foundation of a scalable, maintainable, and resilient microservices architecture.

Aligned
 Independent
 Resilient
 Scalable
 Evolvable

CAPSTONE DOMAIN AND DATA MODEL · CAPSTONE REST API CONTRACTS

· CAPSTONE MESSAGING ARCHITECTURE



CORE ENTITIES

- Customer owns Orders; Order owns OrderItems referencing Products
- Category groups Products; Payment and Shipping each reference an Order



OWNERSHIP DISCIPLINE

- Each service owns its slice of this model -- no cross-service foreign keys
- Cross-service references travel as IDs in events and API payloads, not joins



CONTRACT CONVENTIONS

- Versioned base paths, plural resource nouns, standard HTTP verbs and status codes
- Consistent error response shape across every service



EXAMPLE CONTRACTS

- Order Service: POST /orders, GET /orders/{id}, GET /orders/{id}/status
- Payment Service: POST /payments;
- Inventory Service: GET /inventory/{sku}



EVENT-DRIVEN FLOW

- Order Service publishes OrderCreated; Payment and Inventory consume it independently
- Shipping Service reacts to PaymentConfirmed to begin fulfillment

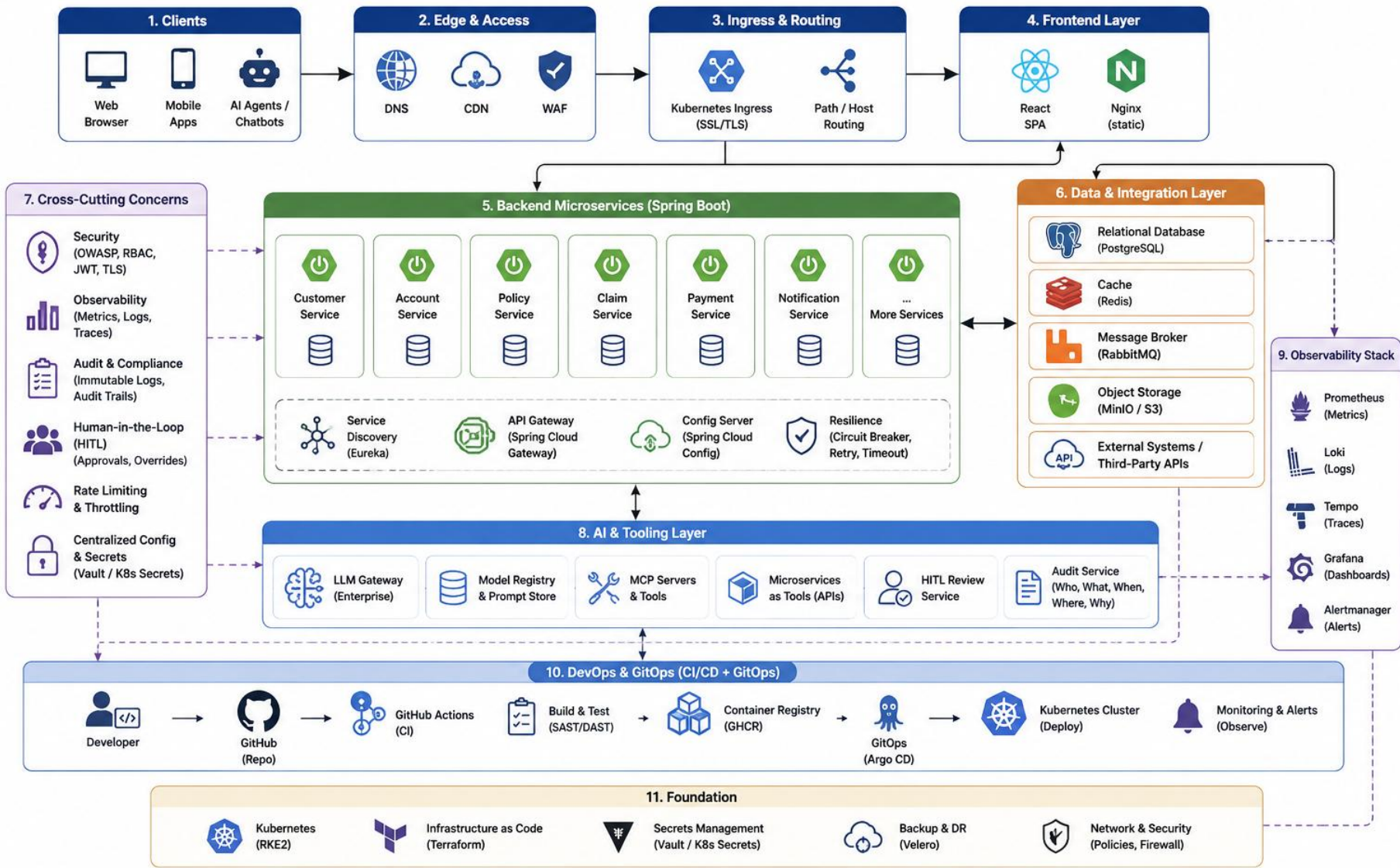


WHY EVENTS, NOT CALLS

- Producers and consumers scale and fail independently
- New consumers (like Analytics or AI Agent services) subscribe without changing producers

KEY TAKEAWAY The core transactional slice of the platform -- Customer, Catalog/Category/Product, Order/OrderItem, Payment, and Shipping - is where most of the capstone's implementation effort concentrates. • A consistent REST contract style across all six core services -- resource paths, verbs, status codes, and error bodies -- keeps the platform predictable for both human and AI consumers. • Kafka topics for order, payment, inventory, and shipping events let services react asynchronously instead of chaining synchronous calls across the order lifecycle.

Capstone Architecture Recap



CAPSTONE DATABASE OWNERSHIP · CAPSTONE TRANSACTION STRATEGY · CAPSTONE CACHING STRATEGY

OWNERSHIP MAP

- Order Service owns order_db; Payment Service owns payment_db; Inventory owns inventory_db
- Customer and Shipping services each own their own database in the same pattern

THE RULE THAT PROTECTS IT

- No shared schemas, no cross-service joins, no direct database access from another service

THE ORDER SAGA

- Order created (pending) -> payment authorized -> inventory reserved -> order confirmed
- A failure at any step triggers compensating actions: release inventory, refund payment, cancel order

WHY A SAGA

- No distributed two-phase commit across independently owned databases
- Each local transaction is followed by an event that triggers the next step

WHAT GETS CACHED

- Product catalog and pricing data with longer TTLs
- Inventory availability with short TTLs, invalidated on every stock change

CACHE DISCIPLINE

- Cache-aside pattern: read cache, fall back to database, populate cache on miss
- Never cache payment or order state -- correctness there outweighs latency

KEY TAKEAWAY Order, Payment, Customer, Inventory, and Shipping each own a private database; any other service that needs their data asks through an API or listens for an event. • Placing an order touches Order, Payment, and Inventory services -- a choreographed saga with compensating actions keeps them consistent without a distributed transaction. • Redis caches read-heavy, slow-changing data like the product catalog, while inventory counts use short TTLs and explicit invalidation so customers never see stale stock.

CAPSTONE RESILIENCE STRATEGY · CAPSTONE SECURITY MODEL · CAPSTONE OBSERVABILITY STRATEGY



RESILIENCE4J POLICIES IN USE

- Timeouts and bounded retries with backoff on every outbound call
- Circuit breakers open on sustained failure; bulkheads isolate thread pools per dependency



GRACEFUL DEGRADATION

- Fallbacks return cached or partial data instead of failing the whole request
- Rate limiting protects services from traffic spikes



LAYERS OF DEFENSE

- OAuth2/OIDC and JWT for user and agent authentication and authorization
- mTLS between services; network policies restrict which services can call which



SECRETS AND ACCESS

- OpenShift Secrets for credentials, keys, and tokens -- never hardcoded
- Role-based access control scoped to least privilege for every service account



THREE PILLARS

- Logs: structured, centralized, correlation IDs on every entry
- Metrics: RED metrics per service, business and technical, alerted on symptoms
- Traces: distributed tracing across every service boundary



OUTCOMES

- Faster mean time to resolution and earlier detection of customer-facing issues

KEY TAKEAWAY Every call between Order, Payment, Inventory, Shipping, and Notification services carries a timeout, a bounded retry, and a circuit breaker so one failing dependency degrades gracefully instead of cascading. • User-facing traffic authenticates through OAuth2/OIDC and JWTs; service-to-service traffic is mutually authenticated; every credential and secret lives in OpenShift Secrets, never in code. • Structured logs, Micrometer/Prometheus metrics, and OpenTelemetry traces, correlated by a shared trace ID, let the team find the root cause of any capstone incident quickly.

CAPSTONE CONTAINERIZATION STRATEGY · CAPSTONE OPENSIFT DEPLOYMENT MODEL · CAPSTONE CI-CD PIPELINE



BUILD PRINCIPLES

- Multi-stage builds: compile in one stage, run from a minimal JRE-Alpine image
- Immutable images, one service per container, non-root execution



OPENSIFT READINESS

- Health checks, externalized config via ConfigMaps and Secrets, small attack surface



DEPLOYMENT SHAPE

- Edge users -> OpenShift Route (TLS, WAF, rate limiting) -> API Gateway -> services
- Data layer (PostgreSQL, MongoDB, Redis) and an integration layer (Kafka, RabbitMQ) sit behind the services



PLATFORM SERVICES USED

- Service mesh, Prometheus, Elastic/Loki, Jaeger/Tempo, Alertmanager, and a secured registry
- Rolling, blue/green, and canary deployment patterns; HPA-based autoscaling



THE NINE STAGES

- Source -> Build -> Quality Gates -> Test -> Package -> Scan -> Push -> Deploy -> Verify & Monitor
- Quality, test, security, and deployment gates block promotion on failure



ENVIRONMENT PROMOTION

- Dev auto-deploys on merge; QA/Staging mirrors production for pre-release validation
- Production releases are controlled, monitored, and alerted end-to-end

KEY TAKEAWAY Every capstone service builds through the same multi-stage Dockerfile pattern -- minimal JDK base image, non-root user, externalized configuration -- so images stay small, secure, and consistent. • External traffic enters through an OpenShift Route with TLS termination and WAF/rate limiting, then a Spring Cloud Gateway routes it to User, Order, Inventory, and Payment services inside a dedicated project namespace. • Source, build, quality gates, test, package, scan, push, deploy, and verify-and-monitor form one pipeline that promotes a change through Dev, QA/Staging, and Production only after every gate passes.

CAPSTONE GIT WORKFLOW · CAPSTONE AUTOMATED TESTING STRATEGY



BRANCH ROLES

- main: production-ready, tagged releases develop: integration, always deployable
- feature/*: new work off develop release/*: stabilization
- hotfix/*: urgent fixes off main



THE PYRAMID

- Unit (40%) and component/service tests (30%) run fast and in isolation
- Integration and contract tests (20%) verify messaging and service interactions
- End-to-end tests (10%) validate critical user journeys across the platform



RULES THAT PROTECT THE HISTORY

- Pull requests and at least one approval required for every merge to develop and main
- All checks must pass; conventional commit messages; no direct pushes to main

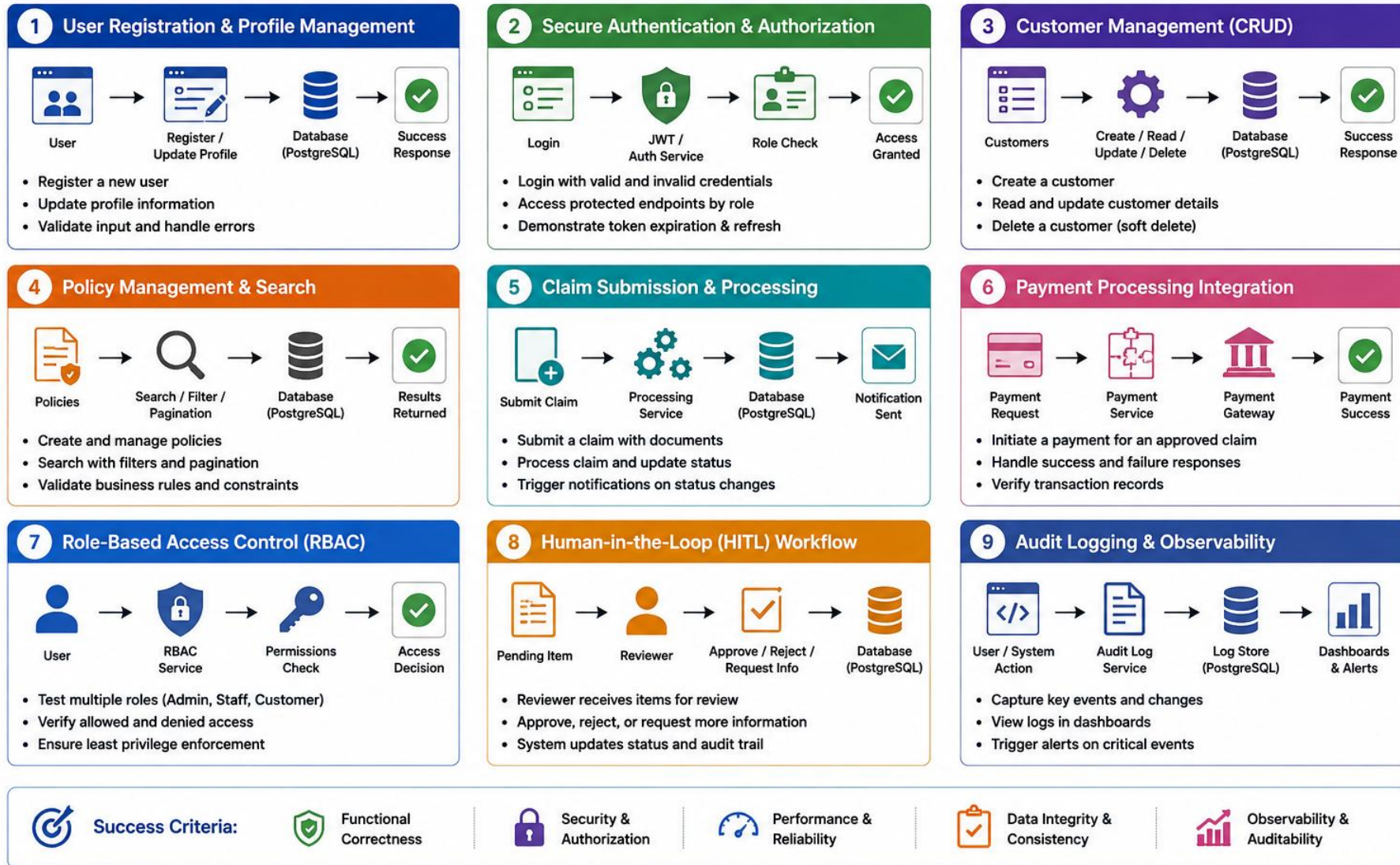


QUALITY GATES

- Coverage thresholds, zero critical/high vulnerabilities, 100% contract-test success required to merge

KEY TAKEAWAY A protected main branch, an always-deployable develop branch, short-lived feature branches, and release/hotfix branches keep the capstone's history clean and every merge reviewed. • Most capstone tests are fast unit and component tests; a smaller layer of integration and contract tests verifies service interactions; a thin end-to-end layer validates full user journeys.

Required Demonstration Scenarios



CAPSTONE GITHUB COPILOT WORKFLOW · CAPSTONE AGENTIC AI WORKFLOW · CAPSTONE MCP SERVER DESIGN



THE SEVEN-STEP WORKFLOW

- Plan & understand -> design & scaffold -> implement -> test -> document -> review & refactor -> commit & integrate
- A continuous feedback loop refines each step as the capstone evolves



NON-NEGOTIABLE GUARDRAILS

- Human review required for all AI-generated code; never commit secrets from a prompt
- Validate correctness, security, and performance before merging



THE SIX-STEP LOOP

- User request -> agent orchestrator (intent, planning) -> tool selection -> execute actions
- Aggregate and reason over results -> generate the final response to the user



GUARDRAILS AND SAFETY

- Least-privilege access to tools; human approval for destructive actions
- Prompt-injection protection, rate limiting, and comprehensive audit logging



MCP SERVER LAYERS

- MCP interface layer: tool discovery, schema management, request routing
- Capabilities registry (tools, resources, prompts) and an adapter layer to backend service APIs



POLICY AND SECURITY ENFORCEMENT

- Authentication and authorization, permission checks, rate limiting, and audit logging at the boundary
- Strongly typed, versioned tool schemas reduce ambiguity for agents

KEY TAKEAWAY A seven-step AI-augmented workflow -- plan and understand, design and scaffold, implement, test, document, review and refactor, commit and integrate -- keeps Copilot as a partner, never an autopilot, on the capstone build. • A six-step agent loop -- user request, orchestrator, tool selection, execute actions, aggregate and reason, response -- lets an AI agent complete real capstone workflows, like cancelling an order and issuing a refund, safely and observably. • Tools, resources, and prompts behind an MCP interface layer -- with authentication, permission checks, rate limiting, and audit logging enforced at the boundary -- let agents discover and invoke capstone operations safely.

CAPSTONE HUMAN-APPROVAL WORKFLOW · CAPSTONE AUDIT AND COMPLIANCE REQUIREMENTS · CAPSTONE FAILURE SCENARIOS



THE FIVE-STEP FLOW

- Agent plans action -> risk evaluation and policy check -> human review (if required) -> execute -> audit, notify, monitor
- Low-risk actions auto-approve and execute; high-risk actions route to a human



BEST PRACTICES

- Define clear triggers by risk and impact; show full context and rationale to the reviewer
- Set review time limits and log every decision for traceability



WHAT EVERY EVENT CAPTURES

- Actor (user or agent), action, resource, timestamp, correlation ID, and result
- Enriched with identity, service, request parameters, and success/failure



COMPLIANCE CONTROLS

- Write-once, append-only storage with encryption at rest and in transit
- Defined retention periods and least-privilege access to audit data



FAILURE CATEGORIES TO TEST

- Service failures: crash, unresponsive pod
- Data layer failures: database down, connection pool exhaustion
- Network issues: latency, packet loss
- Messaging failures: broker down, consumer backlog
- Resource exhaustion: high CPU/memory, disk full, thread pool saturation

KEY TAKEAWAY A risk-evaluation step decides whether an agent's proposed action executes automatically or waits for a human reviewer to approve, request changes, or reject it before anything touches production data. • Every audit event records who did what, when, where, why, and the outcome, written to an append-only store with encryption and retention controls so the platform can withstand a compliance review. • The capstone's resilience design is only proven once it's been tested against real failure scenarios: a crashed pod, an unreachable database, packet loss, a down message broker, or exhausted CPU and memory.



CAPSTONE PERFORMANCE REQUIREMENTS · CAPSTONE PRODUCTION-READINESS CRITERIA · CREATING THE SPRING BOOT SERVICES

EXAMPLE TARGETS

METRIC	TARGET
API latency (p95)	≤ 300 ms
Throughput	$\geq 1,000$ requests/sec
Error rate	$\leq 0.1\%$
Availability	$\geq 99.9\%$

PROJECT SKELETON

- spring-boot-starter-web, -data-jpa, -validation, and -actuator as the baseline dependencies
- Layered packages: controller, service, repository, dto, entity, config

READINESS CHECKLIST

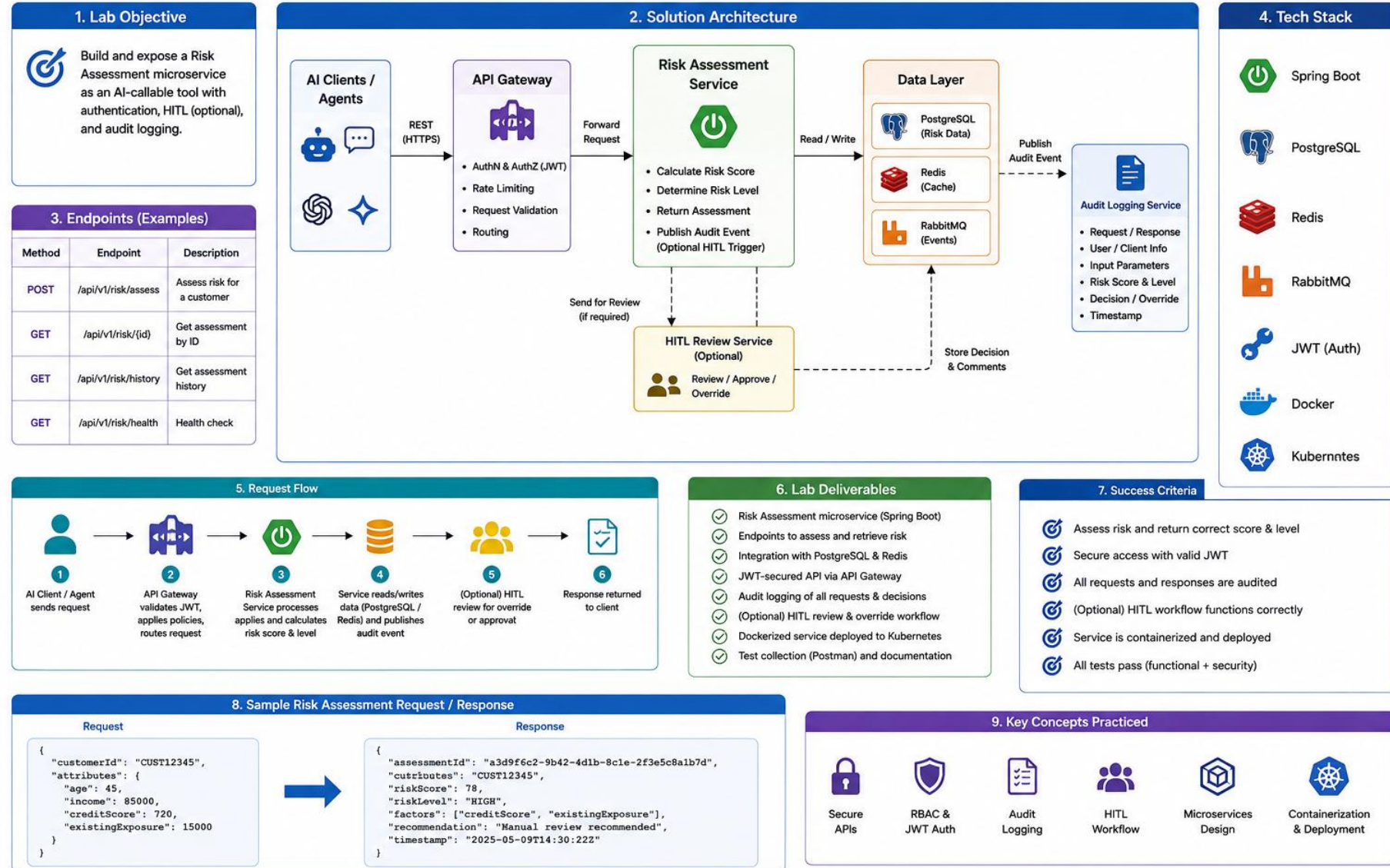
- All automated tests and security scans pass; performance targets are met
- Observability, alerting, and rollback are proven; audit and compliance controls verified
- Runbooks and on-call ownership are documented for every service

CONFIGURATION DISCIPLINE

- application.yml per profile (dev, qa, prod); no environment values hardcoded
- Each service (like OrderService) is generated the same way for consistency

KEY TAKEAWAY Explicit performance targets -- response time percentiles, throughput, error rate, and availability -- give the capstone team a measurable bar instead of a vague sense of "fast enough." • Production readiness for the capstone means every quality gate has passed, every failure scenario has been tested, observability is in place, and the team has a runbook -- not just that the demo works. • Order, Payment, Customer, Inventory, Shipping, and Notification services all start from the same Spring Boot skeleton: Web, Data JPA, Validation, and Actuator starters, layered packages, and profile-based configuration.

Lab 5 Overview - Risk Assessment Service



IMPLEMENTING REST API CONTRACTS · IMPLEMENTING PERSISTENCE AND MIGRATIONS



IMPLEMENTATION PATTERN

- @RestController delegates to a service layer; @Valid enforces request DTO validation
- 201 Created with Location header on POST; 404 with a structured error body on not-found



MIGRATION DISCIPLINE

- Versioned, checked-in SQL migration scripts per service, applied on startup
- spring.jpa.hibernate.ddl-auto=validate in every environment beyond local development



REPOSITORY LAYER

- Spring Data JPA repositories per aggregate root, with indexes matching real query patterns

KEY TAKEAWAY Implementing a contract means a thin @RestController, @Valid request DTOs, a consistent status-code convention, and a shared error-response shape -- the same pattern on every capstone service. • Every capstone service manages its own schema through versioned Flyway or Liquibase migrations, never through Hibernate's auto-DDL, so schema changes are reviewable and repeatable across environments.

IMPLEMENTING SERVICE-TO-SERVICE COMMUNICATION · IMPLEMENTING EVENT-DRIVEN INTEGRATION



THE CALL PATTERN

- WebClient or a Feign-style declarative client per downstream service
- Every call wrapped in @CircuitBreaker, @Retry, and a bounded @TimeLimiter



PRODUCER AND CONSUMER PATTERN

- KafkaTemplate publishes domain events after the local transaction commits
- @KafkaListener consumers are idempotent, tracking processed event IDs



FAILURE HANDLING

- Dead-letter topics and retry policies for poison messages

KEY TAKEAWAY Synchronous calls between capstone services -- Order calling Inventory to check stock, for example -- go through a declarative client wrapped in a timeout, retry, and circuit breaker, never a bare HTTP call. • Order Service publishes OrderCreated with Spring Kafka; Payment and Inventory services consume it with idempotent, at-least-once handlers that tolerate redelivery without double-processing.

IMPLEMENTING DISTRIBUTED DATA CONSISTENCY · ADDING REDIS CACHING · ADDING RESILIENCE4J POLICIES



THE OUTBOX PATTERN

- Order and its OrderCreated event row are written in one local transaction
- A separate relay process publishes outbox rows to Kafka and marks them sent



IMPLEMENTATION PATTERN

- @Cacheable on catalog read methods; @CacheEvict on writes that change cached data
- Redis connection configured with sensible TTLs per cache region



COMPENSATION LOGIC

- Saga compensating handlers implement the rollback steps designed in Topic 659



POLICIES IN CODE

- @CircuitBreaker(name="inventoryService", fallbackMethod="...") around risky calls
- @Retry with exponential backoff; @Bulkhead to isolate thread pools per dependency

KEY TAKEAWAY Writing the order and its outgoing event in the same local transaction -- the outbox pattern -- prevents the classic dual-write bug where a database commit succeeds but the event never gets published, or the reverse. • Spring's @Cacheable and @CacheEvict annotations over a Redis-backed cache manager implement the read-heavy caching strategy for the product catalog, with explicit invalidation on every inventory update. • @CircuitBreaker, @Retry, @TimeLimiter, and @Bulkhead annotations from Resilience4j, configured per downstream dependency, are how the resilience strategy from Topic 661 actually gets enforced at runtime.

ADDING OAUTH2 AND OIDC SECURITY · ADDING LOGS, METRICS, AND TRACES · GENERATING TESTS WITH GITHUB COPILOT



IMPLEMENTATION PATTERN

- spring-boot-starter-oauth2-resource-server validates JWT signature, issuer, and audience
- @PreAuthorize enforces scopes and roles at the method level



EXAMPLE PROMPT

- "Generate JUnit 5 tests for OrderService using Mockito, including edge cases"



INSTRUMENTATION IN CODE

- Structured logging with trace ID, span ID, and service name in every entry
- Micrometer counters and timers exposed via /actuator/prometheus
- OpenTelemetry SDK auto-instruments HTTP, JPA, and Kafka calls



STILL THE DEVELOPER'S JOB

- Verify each assertion is meaningful, not just present
- Add missing edge cases Copilot didn't think to cover

KEY TAKEAWAY Each capstone service is configured as an OAuth2 resource server that validates incoming JWTs, checks scopes and roles, and rejects unauthenticated or under-privileged requests before they reach a controller. • Structured JSON logs with correlation IDs, Micrometer metrics exposed through Actuator, and an OpenTelemetry SDK propagating trace context are how every capstone service actually becomes observable. • A well-scoped prompt asking Copilot to generate JUnit 5 tests for OrderService, including edge cases, gets a fast first draft -- the developer's job is to check every assertion actually verifies the right behavior.

REVIEWING AND CORRECTING AI-GENERATED CODE · PACKAGING SERVICES AS CONTAINERS · BUILDING THE CI-CD PIPELINE



COMMON CORRECTIONS NEEDED

- Deeper validation than a simple null check; complete field mapping
- Explicit transactional boundaries; proper exception handling



THE BUILD IN PRACTICE

- Build stage compiles with Maven; runtime stage copies only the built JAR
- USER spring:spring; EXPOSE 8080; ENTRYPOINT tuned for container memory limits



PIPELINE AS CODE

- Declarative Jenkinsfile with stages matching the nine-stage design
- SonarQube quality gate, Trivy image scan, and OpenShift deployment all wired into the pipeline

KEY TAKEAWAY A null check that's too shallow, a mapper that misses a field, a missing transactional boundary -- Copilot's first draft on a capstone service still needs a human reviewer checking correctness, security, and maintainability. • Applying the containerization strategy from Topic 664 to real code: a multi-stage Dockerfile per service, an OpenJDK-Alpine runtime image, a non-root user, and health checks wired to Spring Boot Actuator. • A declarative Jenkinsfile with build, quality-gate, test, package, scan, push, and deploy stages -- each one able to fail the pipeline -- is how the CI/CD design from Topic 666 becomes a real, running pipeline.

DEPLOYING THE CAPSTONE TO OPENSIFT · SCALING CAPSTONE SERVICES · IMPLEMENTING THE AGENTIC AI WORKFLOW



THE DEPLOYMENT SEQUENCE

- Push image -> deploy to OpenShift -> run database migrations -> health and readiness checks
- Traffic only routes to pods that pass their readiness probe



SCALING MECHANICS

- HPA targets per-service CPU/memory utilization or a custom metric (like Kafka lag)
- Stateless services scale horizontally without session affinity concerns



EXAMPLE: "CANCEL ORDER #1245 AND REFUND"

- Agent understands intent, retrieves order details, calls Order Service to cancel
- Agent publishes a RefundRequested event; Payment Service processes the refund
- Agent confirms the refund and notifies the user; every step is logged and audited

KEY TAKEAWAY Deploying to OpenShift means a Deployment with liveness/readiness probes per service, a Route with TLS for external access, ConfigMaps and Secrets for configuration, and database migrations run before traffic is allowed in. • A Horizontal Pod Autoscaler watching CPU, memory, or a custom queue-depth metric adds or removes replicas of Order, Inventory, and other capstone services automatically as real demand rises and falls. • Implementing the agent workflow from Topic 670 means building the orchestrator that plans and reasons, wiring it to a tool registry (MCP), and enforcing RBAC, rate limits, and human-in-the-loop policies at every step.

EXPOSING CAPSTONE OPERATIONS THROUGH MCP · ADDING HUMAN-IN-THE-LOOP CONTROLS



TOOL DEFINITION IN PRACTICE

- Each tool declares name, description, input schema, output schema, and required fields
- MCP server validates input, authorizes the caller, invokes the backend service, returns a structured result



IMPLEMENTATION PRACTICES

- Define clear triggers by risk and impact (e.g., refunds over a threshold)
- Provide full context, rationale, and a simple UI for the reviewer's decision
- Set SLAs for review response and escalate automatically if they're missed

KEY TAKEAWAY Exposing capstone operations through MCP means wrapping each one -- `create_order`, `get_order`, `update_inventory`, `process_payment`, `send_notification` -- in a strongly typed tool with input/output schemas, least-privilege access, and audit logging. • Adding human-in-the-loop controls in practice means defining which actions auto-approve, giving reviewers full context and a simple approve/request-changes/reject decision, and logging every outcome for audit.

RECORDING AGENT AND USER AUDIT EVENTS · RUNNING END-TO-END TESTS · RUNNING FAILURE AND RECOVERY TESTS



THE AUDIT EVENT SHAPE

- eventId, timestamp, actor (user or agent), action, resource, service, correlationId, result
- Stored write-once in an append-only, encrypted, access-controlled audit store



EXAMPLE E2E SCENARIO

- Place an order from the web portal -> Order and Inventory services validate the transaction
- Payment service processes it; Notification service sends confirmation
- Test verifies order status, inventory update, and notification -- the full business outcome



EXAMPLE FAILURE TESTS

- Kill an Order Service pod and verify automatic restart and traffic recovery
- Simulate a database outage and validate graceful fallback, not data corruption
- Simulate broker downtime and validate dead-letter handling and recovery

KEY TAKEAWAY Every user and agent action writes a structured audit event -- actor, action, resource, timestamp, correlation ID, and outcome -- to an immutable, encrypted, access-controlled store.

- An end-to-end test for the capstone places an order through the UI, then verifies the order and inventory services, the payment service, and the notification service all reached the correct final state.
- Failure and recovery testing on the capstone means deliberately killing a service pod, simulating a database outage, or injecting high latency, then confirming automatic recovery and no data loss.

RUNNING SECURITY VALIDATION · RUNNING PERFORMANCE VALIDATION · DEMONSTRATING BLUE-GREEN OR CANARY DEPLOYMENT



EXAMPLE SECURITY CHECKS

- Scan container images for critical vulnerabilities; verify JWT signature, expiration, audience
- Test API endpoints for authorization bypass; check for SQL/NoSQL injection and XSS
- Ensure secrets are stored in a secure vault, never in code or config files



BLUE/GREEN

- Deploy to Green, run health and smoke tests, switch traffic, keep Blue as instant fallback



TYPES OF PERFORMANCE TESTS

- Load testing (expected traffic) and stress testing (find the breaking point)
- Spike testing (sudden traffic surges) and soak testing (sustained load over time)



CANARY

- Route 10% of traffic to the new version, monitor metrics, then ramp toward 100%

KEY TAKEAWAY Security validation for the capstone runs SAST, DAST, dependency scanning, container scanning, and secrets scanning, then checks the results against OWASP Top 10 risks before anything ships. • Performance validation runs load, stress, spike, soak, and scalability tests against the capstone, then checks the measured p95 latency, throughput, and error rate against the targets set in Topic 675. • A blue/green demo validates a new version in an idle Green environment before switching all traffic to it; a canary demo routes a small percentage of real traffic to the new version and watches metrics before ramping further.

DEMONSTRATING AUTOMATED ROLLBACK · DEMONSTRATING AGENT-TO-MICROSERVICE INTERACTION · CAPSTONE ARCHITECTURE REVIEW

EXAMPLE OUTCOME

- Detected error rate of 6.5% (threshold: 2%) -- automatic rollback executed
- Traffic returned to the previous stable version in under a minute; error rate recovered

SIX-LAYER REVIEW

- Clients & consumers -> ingress & edge (Route, API Gateway, security) -> six core microservices
- Data & messaging layer -> observability & operations -> cross-cutting capabilities (CI/CD, RBAC, DR)

THE DEMO FLOW

- User asks a question -> agent plans and identifies tools -> agent queries MCP for schemas
- Agent invokes tools; MCP routes to Order and Shipping services -> results are aggregated and returned

KEY TAKEAWAYS

- Event-driven, resilient by design, secure by default, observable at scale, AI-ready through MCP

KEY TAKEAWAY A rollback demo deploys a new version, deliberately breaches an error-rate or latency threshold, and shows the pipeline detect it, automatically route traffic back to the previous stable version, and verify recovery. • A live demo traces one user request through the agent, the MCP server's tool discovery and execution, the Order and Shipping services, and back -- showing the whole agentic path is secure, observable, and correct. • Stepping back across the whole platform -- clients, ingress and edge, the Order/Payment/Customer/Inventory/Shipping/Notification microservices layer, data and messaging, and observability -- confirms the architecture is business-aligned, resilient, secure, and AI-ready.

CAPSTONE CODE REVIEW · CAPSTONE DEMONSTRATION AND FEEDBACK



REVIEW FOCUS AREAS

- Correctness and edge cases; security (auth, input validation, secrets)
- Performance (queries, caching); readability, maintainability, and test coverage



DEMONSTRATION FLOW

- Introduction -> live demo -> key scenarios -> AI agent interaction -> observability -> summary



CHECKLIST HIGHLIGHTS

- No hardcoded values; AI-generated code reviewed just as carefully as human-written code

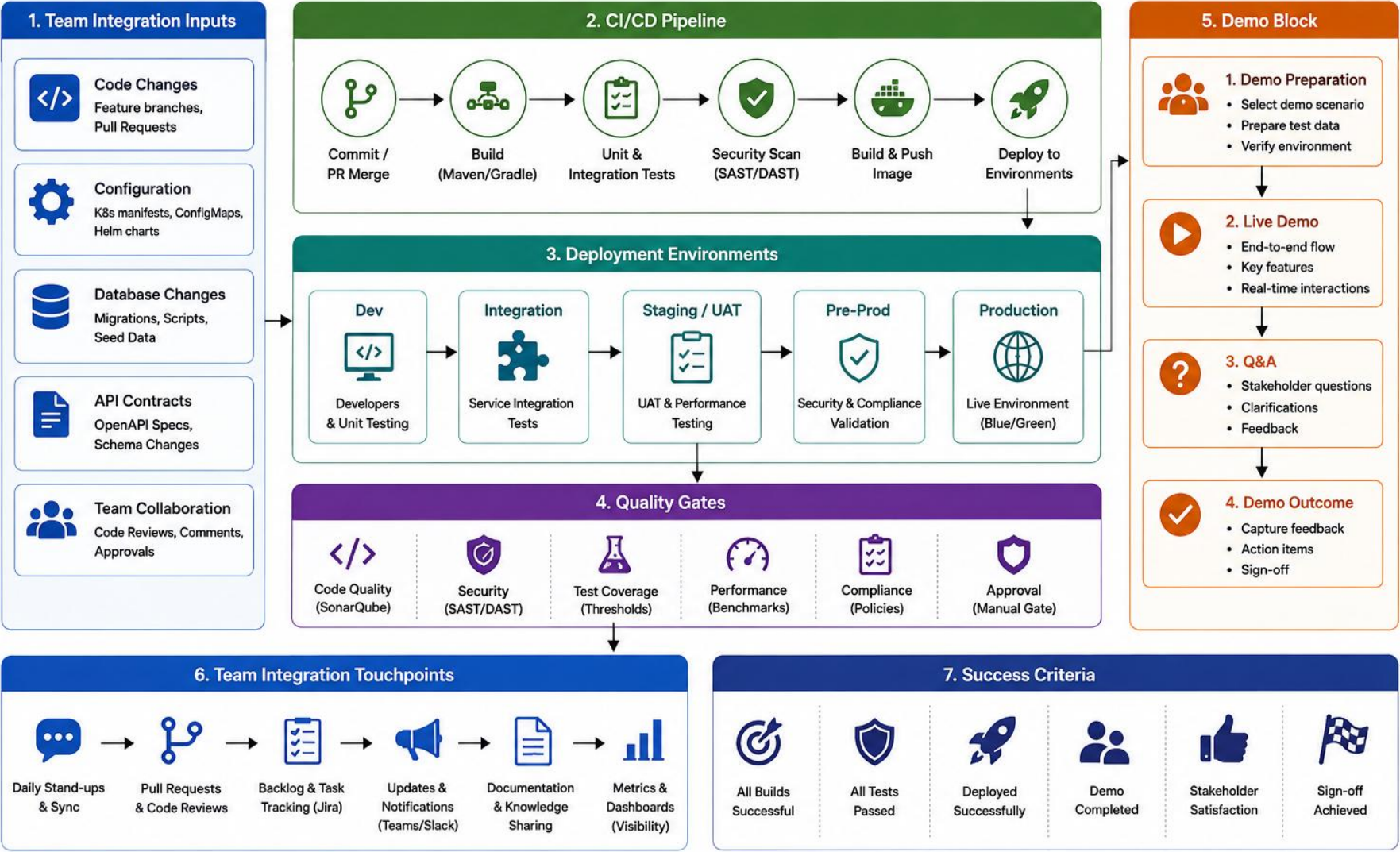


FEEDBACK PROCESS

- Collect feedback -> analyze and prioritize -> create action items -> track and improve

KEY TAKEAWAY A capstone code review walks a real Spring Boot method -- like `OrderService.createOrder` -- checking validation, mapping correctness, transactional boundaries, and response structure before approving the merge. • The final capstone demonstration walks the full solution end-to-end -- introduction, live demo, key scenarios, AI agent interaction, observability, and summary -- then captures structured feedback across functionality, reliability, security, usability, and overall value.

Team Integration and Demo Block



MICROSERVICES FUNDAMENTALS RECAP · SERVICE DESIGN RECAP · SPRING BOOT DEVELOPMENT RECAP



WHAT WE ESTABLISHED

- Why organizations move from monoliths to microservices, and the tradeoffs involved
- Core characteristics: independent deployability, decentralized data, business-capability alignment



WHAT WE ESTABLISHED

- Domain-driven boundaries, bounded contexts, and API contract design
- Versioning, idempotency, and status-code discipline as the law between services



WHAT WE ESTABLISHED

- Production-shaped services from the start: starters, profiles, Actuator, layered packages
- Externalized configuration as a requirement, not an afterthought

KEY TAKEAWAY Microservices trade a monolith's simplicity for independent deployability, technology flexibility, and fault isolation -- at the cost of distributed-systems complexity the rest of this course exists to manage. • Well-drawn service boundaries and disciplined REST API contracts are the design decisions that determine whether a microservices system stays maintainable or turns into a distributed monolith. • The Spring Boot habits built early in this course -- starters, profile-based configuration, Actuator, and externalized config -- are exactly the habits that let a service run unmodified on OpenShift later.

INTEGRATION AND MESSAGING RECAP

Course Recap
Day 5 • Module 29

Connecting services reliably, asynchronously, and at scale while maintaining loose coupling and resilience.



Effective integration patterns and messaging enable microservices to communicate reliably, handle failures gracefully, and stay independent.

1. COMMUNICATION PATTERNS



Synchronous (HTTP)

- REST APIs for request/response
- Simple to implement and understand
- Best for real-time interactions
- Use timeouts, retries, and circuit breakers



Asynchronous (Messaging)

- Queue or topic-based decoupled communication
- Improves scalability and fault tolerance.
- Best for events, notifications, and long-running tasks
- Use idempotency and proper delivery guarantees



Hybrid Integration

- Combine sync and async as needed
- Use API Gateway for external entry
- Choose the right pattern per use case

2. MESSAGING FUNDAMENTALS



Queues (P2P)

Point-to-point messaging between producer and single consumer.



Topics (Pub/Sub)

Publish once, subscribe many. Ideal for events and broadcasts.



Delivery Guarantees

At-most-once, at-least-once, exactly-once (under constraints).



Idempotent Consumers

Ensure safe processing of duplicate messages.



Retries & Redelivery

Handle transient failures with backoff and redelivery policies.



DLQ

Dead-letter queues isolate poison messages for later inspection.



Ordering & Partitioning

Maintain order where needed using partitions and keys.



Security

Secure brokers, topics, and messages with encryption and ACLs.

3. POPULAR MESSAGE BROKERS



- High throughput and scalability
- Topic-based pub/sub
- Partitioning for scale and ordering
- Strong replay and retention
- Great for streaming and analytics

VS



- Flexible routing with exchanges
- Queue-based (P2P) and pub/sub
- Rich routing and binding options
- Simpler for traditional messaging
- Great for task queues and RPC

4. RELIABILITY PATTERNS



Transactional Outbox

Guarantee events are published reliably.



Change Data Capture

Capture changes from DB and publish events.



Saga Pattern

Manage distributed transactions with compensating actions.



Idempotency Key

Prevent duplicate effects from reprocessed messages.

5. OBSERVABILITY FOR INTEGRATION



Metrics

Throughput, lag, errors, retries, and consumer performance.



Logs

Correlation IDs across services and messages.



Traces

End-to-end visibility for request and event flows.



Alerts

Detect failures, DLQ growth, and capacity issues early.

7. OUTCOMES



Loose Coupling

Services evolve independently.



High Scalability

Handle more load and traffic efficiently.



Better Resilience

Systems withstand failures and recover gracefully.



Business Agility

Deliver value faster with reliable integrations.

6. KEY TAKEAWAYS

- ✓ **Choose the Right Pattern**
Select sync, async, or hybrid based on business needs and tradeoffs.
- ✓ **Design for Independence**
Use events and messages to minimize coupling between services.
- ✓ **Plan for Failures**
Implement retries, idempotency, DLQs, and circuit breakers.
- ✓ **Ensure Data Consistency**
Use eventual consistency, outbox pattern, and sagas when needed.
- ✓ **Observe Everything**
Monitor queues, lag, failures, and message rates.
- ✓ **Secure by Design**
Authenticate services and protect messages in transit and at rest.

DATA AND TRANSACTION PATTERNS RECAP · RESILIENCE AND PERFORMANCE RECAP · OBSERVABILITY RECAP



WHAT WE ESTABLISHED

- Database-per-service as the non-negotiable ownership rule
- Sagas and compensating transactions in place of two-phase commit



WHAT WE ESTABLISHED

- Resilience4j patterns for graceful degradation under dependency failure
- Caching and SLO-driven performance budgets as a discipline, not an afterthought



WHAT WE ESTABLISHED

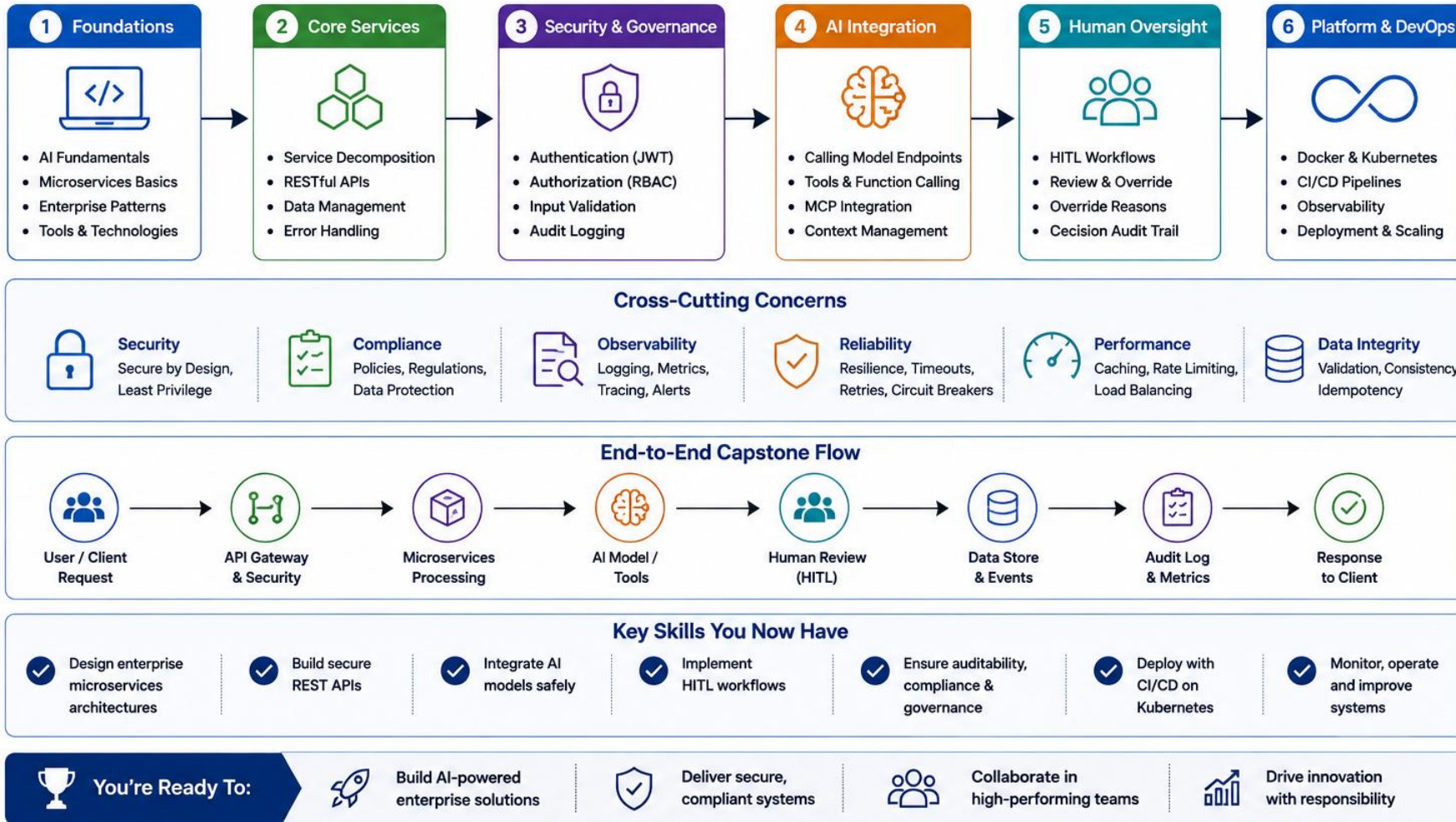
- The three pillars of observability, correlated by trace and span IDs
- SLO-based alerting that signals user impact, not just technical thresholds

KEY TAKEAWAY Database-per-service ownership, sagas with compensating actions, and the outbox pattern are how this course achieved consistency across services without ever needing a distributed transaction. • Resilience4j's timeouts, retries, circuit breakers, and bulkheads, paired with a deliberate caching strategy, are the patterns this course relied on to keep services up when their dependencies fail. • Structured logs, Micrometer metrics, and distributed tracing, all correlated by a shared trace ID, are what this course relied on to turn an incident from a guessing game into a fast, evidence-based investigation.

Course Summary



Outcome: You can design, build, and deploy secure, enterprise-grade AI-powered microservices with authentication, audit logging, and human oversight.



DEVOPS AND CI-CD RECAP · CONTAINERIZATION AND OPENSIFT RECAP · SECURITY AND COMPLIANCE RECAP



WHAT WE ESTABLISHED

- GitOps-style pipelines with build, quality, test, security, and deployment gates
- Environment promotion instead of rebuilding artifacts per environment



WHAT WE ESTABLISHED

- Multi-stage, minimal, non-root container images as the standard build pattern
- OpenShift Routes, probes, ConfigMaps/Secrets, and HPA as the deployment platform



WHAT WE ESTABLISHED

- OAuth2/OIDC, JWT, and mTLS as the security baseline for every service
- Secrets management and audit logging as compliance requirements, not extras

KEY TAKEAWAY An automated CI/CD pipeline with quality, test, and security gates, promoting the same artifact through Dev, QA, and Production, is what this course relied on to make every change a deployable, evidenced release. • A consistent containerization pattern, deployed to OpenShift with routes, probes, ConfigMaps, and autoscaling, is what let every service in this course run identically across dev, staging, and production. • OAuth2/OIDC for users, mTLS between services, and disciplined secrets management are the controls this course relied on to satisfy the audit question every regulated system eventually faces.

GITHUB COPILOT RECAP · AGENTIC AI RECAP · MODEL CONTEXT PROTOCOL RECAP



WHAT WE ESTABLISHED

- Prompt engineering -- role, task, context, constraints, output, quality -- as the skill that matters most
- Human review remains mandatory for every AI-generated line before it ships



WHAT WE ESTABLISHED

- The agent loop: plan, select tools, execute, reason, respond, learn from feedback
- Guardrails -- least privilege, human approval, audit logging -- as non-negotiable design elements



WHAT WE ESTABLISHED

- Tools, resources, and prompts as the three building blocks of an MCP server
- Security and policy enforcement at the MCP boundary, not inside every agent

KEY TAKEAWAY Across this course, Copilot proved fastest as a drafting partner for boilerplate, tests, and documentation -- never as a substitute for the developer's own review, judgment, and final sign-off. • The agentic AI pattern -- understand intent, select tools, execute actions, reason over results, respond -- with human approval gating anything risky, is what this course built toward from the very first microservice. • MCP's tools, resources, and prompts, enforced behind an authentication and policy boundary, are what let this course's AI agents discover and invoke microservice capabilities safely, without hardcoding integration logic.

PRODUCTION-READINESS CHECKLIST · COMMON IMPLEMENTATION PITFALLS · RECOMMENDED MICROSERVICES ENGINEERING PRACTICES



THE CHECKLIST

- All automated tests and security scans pass; performance targets are met
- Observability and rollback are proven; runbooks and ownership are documented



PITFALLS TO AVOID

- Sharing a database across services; synchronous call chains with no timeouts
- Trusting AI-generated code without the same review bar as human code



THE PRACTICES TO KEEP

- Own your data; test your contracts; wrap every dependency in resilience policies
- Instrument everything before you need it, not after an incident

KEY TAKEAWAY A production-readiness checklist -- tests, security, performance, observability, rollback, and runbooks all signed off -- is the objective bar this course used in place of a subjective "it feels done." • The pitfalls that repeatedly undo microservices projects -- shared databases between services, missing timeouts and retries, and skipping review on AI-generated code -- are avoidable with the discipline this course taught. • Database-per-service ownership, contract testing, resilience policies on every call, and full observability are the four practices this course recommends carrying forward into any future microservices project.

TECHNOLOGY SELECTION DECISION GUIDE · FINAL KNOWLEDGE CHECK



HOW TO DECIDE

- Match the technology to the requirement: ordering and replay favor Kafka, routing flexibility favors RabbitMQ
- Strong consistency favors SQL; flexible schema and scale favor NoSQL



WHAT THE CHECK COVERS

- Every module topic from Day 1 fundamentals through Day 5's capstone and AI integration

KEY TAKEAWAY Choosing between Kafka and RabbitMQ, SQL and NoSQL, or synchronous and asynchronous communication is a decision this course teaches you to make from requirements -- throughput, ordering, consistency -- not from trend-following. • The final knowledge check spans every module of this five-day course -- design, Spring Boot, integration, data, resilience, observability, DevOps, security, and AI -- confirming the whole arc from monolith to AI-ready microservices platform landed.

Final Knowledge Check

1

AI & Generative AI Fundamentals



- What are the key differences between traditional AI and Generative AI?
- What are large language models (LLMs)?
- What are tokens, context windows, and prompts?

2

Microservices Architecture



- What are the benefits of microservices?
- How do services communicate?
- What is the role of an API Gateway?

3

Security & Governance



- How does JWT-based authentication work?
- What is RBAC?
- Why are input validation and least privilege important?

4

Calling Models & AI Integration



- How do you call a model endpoint securely?
- What is the purpose of context management?
- How does MCP enhance integrations?

5

Human-in-the-Loop (HITL)



- When should HITL be used?
- What are override reasons?
- How is the audit trail maintained?

6

Tools, MCP & Function Calling



- What is a tool in the context of AI systems?
- How does function calling work?
- How do MCP clients, servers, and tools interact?

7

Observability & Audit Logging



- What key events should be logged?
- Which metrics indicate system health?
- How do dashboards and alerts help operations?

8

Deployment & DevOps



- What are the stages of the CI/CD pipeline?
- How do Docker and Kubernetes support deployments?
- How do you ensure zero-downtime deployments?

9

Performance, Reliability & Scalability



- How do caching and rate limiting improve performance?
- What strategies improve resilience?
- How do you plan for horizontal scaling?

10

Capstone Architecture & End-to-End Flow



- What are the main components of the capstone architecture?
- Walk through the end-to-end request flow.
- How does the system ensure security, auditability, and human oversight?

Capstone Scenarios

 User Registration & Profile Management

 Payment Processing Integration

 Secure Authentication & Authorization (RBAC)

 Risk Assessment Service (AI-Powered)

 Customer Management (CRUD)

 HITL Review & Override

 Policy Management & Search

 Audit Logging & Observability

 Claim Submission & Processing

 End-to-End Integration & Demo

Success Criteria



Functionality

All scenarios work as expected



Security

Authentication, RBAC, validation in place



Reliability

Resilient, fault-tolerant, and available



Performance

Meets response time and load requirements



Auditability

Complete logs and audit trails



User Experience

Clear, intuitive, and responsive



DevOps Excellence

Automated, repeatable, and observable



Capstone Readiness

Demo-ready with documentation

 **You're Ready!**

Apply your knowledge. Build with confidence. Deliver enterprise-grade AI-powered solutions.

 **Next Step:** Complete the capstone demo and showcase your solution!

© 2026 by Innovation In Software Corporation

111

INDIVIDUAL AND TEAM ACTION PLANS

Turn learning into results. Define your next steps and commit to building better microservices solutions in your organization.



INDIVIDUAL ACTION PLAN



1. GOALS

What do I want to achieve as a result of this course?

- _____
- _____
- _____



2. APPLY & PRACTICE

What concepts or skills will I apply first?

- _____
- _____
- _____



3. PROJECTS & INITIATIVES

What projects or improvements will I contribute to?

- _____
- _____
- _____



4. LEARNING & GROWTH

What will I continue learning or explore next?

- _____
- _____
- _____



5. COMMITMENT

When will I take action and how will I track progress?

- _____
- _____
- _____



My Commitment

I will apply what I learned to deliver value, collaborate effectively, and continuously improve.

Target Date: _____



TEAM ACTION PLAN



1. TEAM GOALS

What outcomes do we want to achieve together?

- _____
- _____
- _____



2. KEY INITIATIVES

What initiatives will we prioritize?

- _____
- _____
- _____



3. ROLES & RESPONSIBILITIES

Who will do what?

- _____
- _____
- _____



4. SUCCESS MEASURES

How will we measure success?

- _____
- _____
- _____



5. TIMELINE & MILESTONES

Key milestones and target dates

- _____
- _____
- _____



Team Commitment

We will collaborate, communicate, and deliver resilient, secure, and scalable microservices solutions.

Target Date: _____

PLAN FOR SUCCESS

- ✓ Start small and iterate incrementally.
- ✓ Align with business priorities.
- ✓ Leverage your team and community.
- ✓ Use automation and good practices.
- ✓ Measure, learn, and adapt.
- ✓ Share knowledge and mentor others.



FOCUS AREAS FROM THE COURSE



Service Design

Design for business value, loose coupling, and independent deployment.



Security

Secure by design with strong identity, secrets management, and least privilege.



Resilience

Use resilience patterns to handle failure and maintain availability.



Observability

Implement logs, metrics, and traces for insights and faster troubleshooting.



DevOps

Automate CI/CD, testing, and deployments for speed and quality.



AI & MCP

Use AI tools and MCP to accelerate innovation responsibly.



Your knowledge is powerful.
Your actions create impact.



Commit today. Act consistently.
Build microservices that make a difference.

ADDITIONAL LEARNING RESOURCES · COURSE SUMMARY · QUESTIONS AND CLOSING DISCUSSION



WHERE TO GO NEXT

- Official framework and platform documentation for Spring Boot and OpenShift
- The capstone repository itself, as a working reference for every pattern covered



THE ARC OF THE COURSE

- Design and build services -> integrate and secure them -> operate and observe them -> extend them with AI
- The capstone proved every pattern together, on one real platform



CLOSING THE COURSE

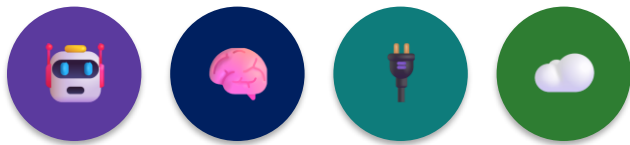
- Open floor for questions on any module, the capstone, or how to apply this at work
- Thank you for completing MD287 -- go build resilient, secure, AI-ready microservices

KEY TAKEAWAY Official Spring, OpenShift/Kubernetes, and OWASP documentation, alongside the capstone codebase built during this course, are the resources to return to when a real project raises a question this course only had time to introduce. • This course took the platform from a single Spring Boot service to a resilient, secure, observable, containerized, AI-ready microservices platform running on OpenShift -- the full arc this capstone demonstrated end-to-end. • An open discussion for remaining questions, edge cases from your own projects, and any part of the microservices, DevOps, or AI-agent material worth revisiting before applying it at work.

LAB 5

Lab 5 — Risk Assessment Service

Risk Assessment Service + OpenShift AI integration; capstone demonstration.



LAB 5 — RISK ASSESSMENT SERVICE — OVERVIEW

Day 5 lab: what you build and how it grows the capstone.



WHAT YOU WILL FINISH WITH

- Risk Assessment Service on port 8083
- Its own database (risk_db on host port 5435)
- A consumer on Kafka topic transactions.submitted (group risk-assessment-service)
- Model call to a classroom stand-in for OpenShift AI (<http://localhost:8090/v1/score>)
- Policy outcomes APPROVE, HOLD, DECLINE
- Model timeout or HTTP 500 → HOLD MODEL_UNAVAILABLE (never auto-approve)
- Human review on HOLD only (risk.write)
- Audit fields: model name/version, score, policy version, correlation id
- Tests for policy, fallback, and JWT

KEY TAKEAWAY Lab 5 builds Risk Assessment Service -- the working code behind today's concepts, and the next layer of the banking capstone.

LAB 5 — RISK ASSESSMENT SERVICE — BEFORE YOU START

Check these before the lab clock starts.

 KNOWLEDGE YOU NEED (FROM DAY 5)

Day 5 idea	How it appears in this lab
Model endpoint	ModelClient POST with Bearer API key, 2s connect / 3s read timeout
Safe fallback	Timeout or 5xx → HOLD, not APPROVE
Deterministic policy	Java PolicyEngine owns APPROVE / HOLD / DECLINE
Human review	HOLD rows stay PENDING until a reviewer posts APPROVE or DECLINE
Audit	Persist modelName, modelVersion, modelScore, policyVersion, correlationId
MCP	Exercise 5.3 worksheet only — auth, HITL, audit if a tool wrapped this API

KEY TAKEAWAY Settle prerequisites and environment first -- most lab time is lost to setup, not to code.

LAB 5 — RISK ASSESSMENT SERVICE — LAB STEPS

8 steps, worked in order.

LAB STEPS

- Step 1 — Start backing services and the mock model
- Step 2 — Confirm the starter still refuses to assess
- Step 3 — Implement PolicyEngine (Exercise 5.2)
- Step 4 — Safe model fallback

LAB STEPS (CONTINUED)

- Step 5 — Protect the API (scopes)
- Step 6 — Run the service and publish sample events
- Step 7 — Optional: live event from Transaction Service
- Step 8 — OpenShift YAML and MCP (Exercise 5.3)

KEY TAKEAWAY Work the steps in order -- each one builds on the last.

LAB 5 — RISK ASSESSMENT SERVICE — SUCCESS CRITERIA

You are done when every box below is ticked.

SUCCESS CRITERIA

- mvn test passes
- Sample events produce APPROVE / HOLD / DECLINE / MODEL_UNAVAILABLE / HIGH_VALUE
- Duplicate event does not create a second row
- Reviewer can complete HOLD; ops cannot

SUCCESS CRITERIA (CONTINUED)

- Logs stay synthetic and secret-free
- OpenShift YAML reviewed (or applied)
- MCP worksheet filled

KEY TAKEAWAY Tick every criterion before moving on -- the next lab builds directly on this one.

DAY 5 COMPLETE

OpenShift AI, MCP, and Capstone Completion

Day 5 complete — Risk Assessment Service + OpenShift AI integration; capstone demonstration.

